

The 2016 and 2017 QBF solvers evaluations (QBFEVAL'16 and QBFEVAL'17)

Luca Pulina^{a,*}, Martina Seidl^b

^a University of Sassari, Italy

^b Johannes Kepler Universität Linz, Austria

ARTICLE INFO

Article history:

Received 6 April 2018

Received in revised form 21 February 2019

Accepted 8 April 2019

Available online 11 April 2019

Keywords:

Quantified Boolean formulas

QBF competition

QBF solving

ABSTRACT

After a break of about five years, in 2016 the classical QBFEVAL has been revived. QBFEVAL is a competitive evaluation of solvers for quantified Boolean formulas (QBF), the extension of propositional formulas with existential and universal quantifiers over the propositional variables.

Due to the enormous interest in QBFEVAL'16, more recently, QBFEVAL'17 was organized. Both competitions were affiliated to the respective editions of the International Conference on Theory and Applications of Satisfiability Testing (SAT'16 and SAT'17), the major conference in research on SAT and related areas.

In this paper we report about the 2016 and 2017 competitive evaluations of QBF solvers (QBFEVAL'16 and QBFEVAL'17), the two most recent events in a series of competitions established with the aim of assessing the advancements in reasoning about QBFs. This report gives an overview of the setup of these two events, on their participants and on the results of the experiments that were performed for evaluating the participating systems.

© 2019 Elsevier B.V. All rights reserved.

1. Introduction

Competitive events in the field of Boolean reasoning have influenced related research agendas and shaped the course of tool developments. Such competitions not only compare state-of-the-art systems, but they also select benchmark sets that challenge the researchers in the respective fields. Nowadays, evaluations are popular for several subfields of Boolean reasoning, including propositional satisfiability (SAT) [1,2], satisfiability modulo theory (SMT) solving [3], and quantified Boolean formulas (QBF) [4].

This paper summarizes the two most recent QBF competitions, QBFEVAL'16¹ and QBFEVAL'17. Those are the last events in a series of competitions that have been established with the aim of assessing the advancements in the field of QBF reasoning and related research. The QBFEVAL events have a long tradition in the relatively young field of QBF research. Already in 2004 the first QBFEVAL competition was successfully organized. For almost 10 years, QBFEVAL was then organized annually or at least bi-annually. After a break of about five years, during which two QBF Gallery events [6,7] took place, QBFEVAL was re-established in 2016. Traditionally, the QBFEVAL events are affiliated with the *International Conference on Satisfiability Testing* (SAT)² during which the winners of the competition are announced and certificates are handed over to the participants. In

* Corresponding author.

E-mail addresses: lpulina@uniss.it (L. Pulina), martina.seidl@jku.at (M. Seidl).

¹ For a preliminary report on QBFEVAL'16 see [5].

² <http://www.satisfiability.org>.

Table 1
QBFEVAL'16 and QBFEVAL'17 at a glance.

Track	QBFEVAL'16		QBFEVAL'17	
	# Systems	# Formulas	# Systems	# Formulas
Prenex CNF	24	825	30	523
Prenex non-CNF	8	890	8	320
2QBF	21	305	29	384
Evaluate & Certify	5	825	—	—
Solver Portfolio	3	825	—	—
Parallel QBF Solvers	6	825	1	✗
Random QBFs	21	580	30	505
Incremental Track	1	✗	—	—
Preprocessing	—	—	2	✗
DQBF	—	—	3	✗

✗... canceled, — ... not announced

addition, details on the organization were presented at SAT-affiliated workshops, in particular the *International Workshop on Quantified Boolean Formulas* in 2016 and the *International Workshop on Pragmatics of Constraint Reasoning* in 2017. QBF researchers were invited by the QBFEVAL organizers (Luca Pulina in 2016 and 2017, and Martina Seidl in 2017) about six months before the SAT conference to submit QBF tools and benchmarks for participation in the competitions. In an evaluation phase, the tools are tested if they are in accordance with the requirements like conformance to the format standards. All experiments were conducted on the StarExec infrastructure [8].³

The huge amount of submissions (44 systems have been submitted to QBFEVAL'16, the 9th QBFEVAL event) indicated the current vitality of research on QBF reasoning tools and motivated the organization of the 10th edition of QBFEVAL, namely QBFEVAL'17, only one year after QBFEVAL'16. In QBFEVAL'17, 47 systems submitted by 19 teams participated. Furthermore, in the two years more than 8.000 novel benchmark formulas were provided. In QBFEVAL events the benchmark selection is done by the organizers in accordance with a team of judges (Hubie Chen, Martina Seidl, Christoph Wintersteiger in 2016 and Olaf Beyersdorff, Daniel Le Berre, Martin Suda, Christoph Wintersteiger in 2017). The role of the judges was further to monitor the whole events in order to guarantee a fair implementation of the competitions. All results and benchmarks are available at

<http://www.qbfeval.org>

The paper is structured as follows. In Section 2 we briefly describe the design of last two QBFEVAL events. In particular, we present the different tracks that were organized, we survey the systems that participated in the competitions, and we discuss how the formulas that we used to construct the benchmark sets were selected. Section 3 and Section 4 present the results of QBFEVAL'16 and QBFEVAL'17. We conclude this paper with a short analysis of the results and an outlook to future editions of QBFEVAL.

2. Setup of QBFEVAL'16 and QBFEVAL'17

In this section, we describe the setup of QBFEVAL'16 and QBFEVAL'17. Both events featured multiple tracks in order to evaluate different aspects of QBF reasoning tools. While some tracks could attract a huge number of participants, other tracks had to be canceled because the number of submissions was too low. After the presentation of the tracks in Section 2.1, we explain how the benchmarks were selected for the various tracks in Section 2.2. Finally, in Section 2.3 we present an overview of the participants of the two QBFEVAL events.

2.1. Tracks of QBFEVAL'16 and QBFEVAL'17

Traditionally, the QBFEVAL events consist of multiple tracks in which different features of QBF solving tools are evaluated. In the call for participation of QBFEVAL'16 eight tracks were announced. Out of these eight tracks only one was canceled (the track on incremental solving), because only one system was submitted. In 2017, seven tracks were proposed in the call and four tracks took place. Table 1 gives an overview on the tracks of QBFEVAL'16 and QBFEVAL'17 w.r.t. the number of participants and also w.r.t. the number of selected formulas. In the following, we shortly summarize the main intentions behind the different tracks.

- In the **Prenex CNF (PCNF)** track systems that are able to process formulas in prenex conjunctive normal form are evaluated, i.e., the considered QBFs are of the structure $\Pi.\psi$ where Π is the quantifier prefix over the variables that

³ <https://www.starexec.org>.

occur in the propositional formula ψ that is in conjunctive normal form (CNF). A propositional formula is in CNF if it is a conjunction of clauses. A clause is a disjunction of literals (variables or negations of variables). The PCNF track may be considered as the main track of the QBFEVAL, because most available solvers are build for this input format and QBF research has a strong focus on the clausal representation of formulas. In consequence, many automated reasoning tasks are encoded in PCNF. Systems and benchmarks in this track have to be conformant to QDIMACS 1.1⁴ input format.

- The **Prenex Non-CNF (PNCNF)** track is devoted to evaluate solvers supporting the QCIR input format for prenex non-CNF formula instances.⁵ In this format, the QBFs have still the structure $\Pi.\psi$ but now the propositional formula ψ is not required to be in CNF. Instead it may contain arbitrary nestings of conjunctions, disjunctions, and negations. Therefore, no structural information contained in the original reasoning problem is preserved and may be exploited by the solvers.
- The **2QBF** track consists of PCNF formulas whose prefix contains only a single alternation of the form $\forall\exists$. The required input format is again QDIMACS 1.1. 2QBF formulas are of interest because already with this very flat quantifier structure, several practical synthesis and verification problems can be encoded. Further, it allows for the application of dedicated solving techniques for which the generalization to an arbitrary prefix structure is not known (yet).
- The **Evaluate & Certify (EC)** track aims to access the state of the art on certification of QBFs in PCNF. In this track, the participating solvers have to produce a certificate that is easy (i.e., compared to solving) to validate. In contrast to SAT, where solvers are required to produce certificates to witness the unsatisfiability of a formula, in QBF only few solvers support the generation of certificates. Due to the limited numbers of participants, this track was not announced in 2017.
- For the **Solver Portfolio (SP)** we invited systems that implement a portfolio approach, i.e., these systems analyze the given formula and based on certain properties of the formula a solver and a configuration for this solver is selected form a pool of solvers and configurations. As the interest in such a track was rather small in 2016, we did not organize it in 2017.
- The aim of the track on **Parallel Solving (PS)** is to assess systems that exploit modern computer architectures and use multiple CPUs for solving. In 2016, we announced this track as a non-competitive track, and six systems participated. These systems were allowed to accept formulas in QDIMACS and/or QCIR format. However, in 2017, only one solver was submitted. Therefore, the PS track was canceled in 2017.
- The formulas included in the **Random QBF (RQBF)** track are generated according to some random model. We did not necessarily require that the formulas are generated according to some clausal random model, but the generation involves one random component at least. The required input format is again QDIMACS 1.1.
- The track on **Incremental Solving (IS)** aims at the evaluation of incremental QBF solvers. Incremental solving is important for applications where the originally satisfiable/unsatisfiable formula is refined by additional clauses/cubes until it becomes unsatisfiable/satisfiable. As in SAT solving, incremental solving capabilities are provided via an API. However, as only one solver has been submitted to this track in 2016, it was canceled. In 2017, the track was not even announced.
- The aim of the **Preprocessing** track announced in 2017 was to evaluate (and award) the impact of preprocessing in the QBF solution process. Nowadays, many solvers that do not explicitly exploit structure in their search process, considerably benefit from extra preprocessing phases. Hence, many solvers include external preprocessing tools. In this track submissions of both preprocessors and solvers without any preprocessing systems were invited. Both preprocessors and solvers were expected to accept input in the QDIMACS 1.1 format. Since only two preprocessors were submitted, the track was canceled.
- Most recently, an increasing interest in DQBF solving could be observed motivated by the demand for efficient reasoning on bit-vectors in the context of SMT. DQBF generalizes QBF by allowing for Henkin quantifiers, lifting the decision problem from PSPACE to NEXPTIME. In order to support this stream of research, the idea was to offer a dedicated track on **DQBF** solving. However, this was too early in 2017. Only three systems (two complete systems and one preprocessor) have been submitted to the DQBF track, therefore, the track was canceled.

2.2. The dataset

In this section we describe the instances selection procedure for both the editions of QBFEVAL. Note that the selection procedure considerably changed from 2016 to 2017. Several requirements on the considered formulas make the selection very challenging. First of all, due to limited resources, not all formulas collected in the QBFLIB can be included for the competition. Second, the selection should be diverse enough that it is not biased towards a certain solving technique. Third,

⁴ <http://www.qbflib.org/qdimacs.html>.

⁵ <http://qbf.satisfiability.org/gallery/qcir-gallery14.pdf>.

the selected instances should not be too easy and also not be too hard such that a comparison of the participating solvers is possible and interesting research challenges are posed.

2.2.1. Dataset of QBFEVAL'16

In 2016, the following three new formula families were submitted:

- **Generalized Tic-Tac-Toe** [9]: 180 formulas in QDIMACS 1.1, submitted by Diptarama, C. Jordan, and A. Shinohara,
- **Random QBFs** [10]: 60 formulas in QDIMACS 1.1 and QCIR format, submitted by F. Ricca, G. Amendola and M. Truszczynski
- a QBF generator for formulas related to the rewriting algorithm described in [11], implemented and submitted by T. Peitl.

The dataset used in the QBFEVAL'16,⁶ consists of four different classes of formulas:

PCNF formulas These formulas were used for the PCNF track, the Evaluate & Certify track, the Solver Portfolio track, and Parallel Solving track. All of the formulas are of fixed structure in prenex conjunctive normal form in QDIMACS 1.1. From each family available in the QBFLIB we randomly selected up to 10 formulas plus 10 formulas from newly submitted families, resulting in a total amount of 825 formulas.

Prenex non-CNF formulas This set consists of formulas represented in the QCIR format that was introduced for the QBF-Gallery 2014 [6,7]. As there were no new submissions, the non-prenex non-CNF dataset of QBFEVAL'10 [12] (after prenexing, 478 formulas were converted to QCIR) was reused. Furthermore, 50% of the formulas of the PCNF set were converted from QDIMACS to QCIR, resulting in a total amount of 890 formulas.

2QBF formulas For the 2QBF track, we selected up to 50 formulas from the families of the QBFLIB that contain the required $\forall\exists$ prefix. This resulted in a dataset of 305 formulas.

Random QBFs Finally, the dataset for the RQBF track includes PCNF formulas that have a probabilistic component in their generation. The dataset used in QBFEVAL'16 includes 580 formulas, 320 of which have been selected from QBFLIB. Another 200 formulas have been generated by the tool BLOCKSQBF [13]—based on the model described in [14]. Finally, all 60 newly submitted random formulas were included.

Discussion The selection process of the dataset for QBFEVAL'16 was completely based on a random selection of the formulas. In particular, no information on runtimes and results from previous competition was used. It was not checked if the formulas could be solved by popular preprocessors. This gave those solvers an advantage that included a third party preprocessor, because it turned out that about one third of the formulas of the PCNF track could be solved by preprocessing alone. Also no information on the formula structure was considered for the selection process, except for the family classification in the QBFLIB. In 2017, we completely changed the selection process as we describe below.

2.2.2. Dataset of QBFEVAL'17

As detailed in the following, QBFEVAL'17 received more than 4000 new instances related to different application domains (in parenthesis the track in which they have been involved):

- **2QBF Encoding of Boolean Functional Synthesis**: 42 instances submitted by S. Akshay, S. Chakraborty, A. K. John, S. Shah and M. N. Rabe, UC Berkeley. (PCNF, 2QBF)
- **Patch Generation for Engineering Change Order of Integrated Circuits**: 5 instances submitted by L. Chen and J. R. Jiang, National Taiwan University. (PCNF)
- **Mapping User-Specified Functions to Configurable Combinational Logic in FPGAs**: 12 instances submitted by T. Preusser, University of Texas. (PCNF)
- **Safety Synthesis using QBF**: 1354 instances submitted by L. Tentrup, Saarland University. (PCNF, PNCNF, 2QBF)
- **Sketch Performance Benchmarks**: 14 instances submitted by M. N. Rabe, UC Berkeley. (PCNF, 2QBF)
- **Bounded Synthesis for Petri Games**: 360 instances submitted by J. Hecking-Harbusch, Saarland University. (PNCNF)
- **Combinational Equivalence**: 50 instances submitted by W. Klieber, Carnegie Mellon University. (PNCNF, RQBF)
- **Hard 2QBFs**: 1873 instances submitted by G. Amendola and F. Ricca, University of Calabria, and M. Truszczynski, Kentucky University. (2QBF, RQBF)
- **QBF Benchmark for Positional Games**: 312 instances submitted by V. Mayer-Eichberger and A. Saffidine, University of New South Wales, Sydney. (RQBF)

⁶ All formulas used in QBFEVAL'16 can be downloaded from <http://www.qbflib.org/eval16.html>.

Table 2

Features of a selection of instances in QBFLIB.

Feature	Min	1st	Med	3rd	Max
Existential Variables	36	1,021	4,806	18,295	2199,062
Universal Variables	1	16	66	180	55,022
Clauses	65	318	15,627	60,578	5934,890
Quantifier Blocks	2	3	3	3	1,141

In order to rely less on random choices, in 2017 we used a different selection scheme w.r.t. 2016. Instead of randomly selecting formulas from a pool of pre-defined families, we took information from the formulas structure as well as results from previous competitions.

PCNF formulas The considered dataset has been composed considering both the results of past QBFEVALs (if applicable) and instance features. For this purpose, we define the empirical hardness coefficient (HC_i) for an instance i as

$$HC_i = \frac{S_i}{S_t} \quad (1)$$

where S_i is the number of solvers that solved i , while S_t is the total number of solvers participating to a given contest.

For the instances selection from the QBFLIB, we used the following approach. Starting from the 15,019 fixed structured instances in QBFLIB with correct QDIMACS format, we discarded

- SAT instances (284);
- instances with a total amount of variables less than 50 (211);
- very easy formulas ($HC_i = 1$) from past QBFEVALs (909).

The next step consisted in classifying the remaining formulas (13,617) by common syntactic features. For each instance, we computed the total amount of existential and universal variables, clauses, and quantified sets. These data have been used to build a dataset in order to group instances with the help of an unsupervised classification algorithm.

In Table 2 we report the five numbers of distributions of the features mentioned above. The table indicates that the data is not normally distributed, so we choose *Partition Around Medoids* (PAM) as unsupervised classification algorithm.

An issue in using PAM (as in most clustering algorithms) is to determine the total amount of clusters. Therefore we tested the number of clusters ranging from 2 to 40. In choosing the final number of clusters, first we discarded results having more than about 30% of instances. After that, we chose the configuration having the highest median of its *silhouette*. The silhouette value [15] measures the similarity of an object to its cluster compared to other clusters and indicates how good or bad this object matches to its cluster. The silhouette ranges from -1 to 1 . A value indicates that the object fits well to its own cluster, but badly to other clusters. We finally set the number of clusters to 32, with a silhouette value equal to 0.481.

In order to understand how the obtained clusters are made, we labeled the feature dataset with the cluster name. This enabled us to classify instances using a supervised learning algorithm. In particular, our choice fell to JRIP, the WEKA [16] implementation of Repeated Incremental Pruning to Produce Error Reduction (RIPPER), a propositional rule learner that generates a symbolic model in the form of rules. Finally, the selection was made picking up 10 instances per cluster and discarding instances when families were over represented.

At the end of the selection process, we obtained a dataset containing 523 instances, and it is composed as follows⁷:

1. 227 formulas of the PCNF track of QBFEVAL'16 having HC in the range $[0, 0.5)$.
2. 21 hard instances, i.e., instances i with $HC_i = 0$ from QBFEVAL'10.
3. 52 (out of 520) hard instances of QBFEVAL'08.
4. 152 formulas from QBFLIB that were never involved in previous QBFEVALs.
5. 70 newly submitted formulas.
6. 1 additional formula from a submitter's suggestions (we received only two "suggestions", but the remaining 9 formulas were already involved in the selection).⁸

Prenex non-CNF formulas The dataset is composed of 515 instances in total. Of these, 385 formulas were already included in the Prenex Non-CNF track of QBFEVAL'16 having hardness coefficient in the range $[0, 0.5)$. The rest of the formulas originates from new benchmark suits, including 40 formulas from the ART family, 42 formulas from Bounded Synthesis for Petri Games, 30 formulas from Combinatorial Equivalences and 38 formulas from Safety Synthesis (the latter both are the same as the formulas in the PCNF track).

⁷ All formulas used in QBFEVAL'17 can be downloaded from <http://www.qbflib.org/eval17.zip>.

⁸ We invited all participants to suggest five instances that will be included in the competition.

Table 3

Systems participating in QBFEVAL'16 and/or QBFEVAL'17.

Solver	Tracks 2016							Tracks 2017				Author(s)
	1	2	3	4	5	6	7	1	2	3	7	
AIGSOLVE	✓							✓			✓	C. Scholl, F. Pigorsch
AQUA	✓						✓					P. Marin
AQME					✓							L. Pulina, A. Tacchella
AREQS			✓									M. Janota
ASPQ			✓							✓		G. Amendola, C. Dodaro, F. Ricca
CADET			✓							✓		M. N. Rabe
CAQE	✓			✓	✓	✓	✓	✓		✓	✓	L. Tentrup, M. N. Rabe
CHEQ				✓								M. Narizzano, C. Peschiera, L. Pulina, A. Tacchella
CUED								✓		✓	✓	L.-C. Chen, J.-H. R. Jiang
DEPQBF	✓		✓				✓	✓		✓	✓	F. Lonsing
DEPQBF-CERT				✓								F. Lonsing
DYNQBF			✓					✓		✓	✓	G. Charwat, S. Woltran
GHOSTQ	✓	✓	✓				✓	✓	✓	✓	✓	W. Klieber
HERETIQ								✓		✓	✓	V. Hadzic
HIQQUER	✓		✓			✓	✓					A. Van Gelder, S. Wood
HORDEQBF						✓						T. Balyo, F. Lonsing
HQSPRE_SOLVER								✓			✓	R. Wimmer, S. Reimer P. Marin, B. Becker
IJTIHAD								✓		✓	✓	V. Hadzic
I _{PROVER} -QBF	✓		✓				✓	✓		✓	✓	K. Korovin
MPIDEPQBF						✓						C. Jordan, L. Kaiser, F. Lonsing, M. Seidl
PAR-PD-DEPQBF						✓						U. Egly, F. Lonsing and J. Oetsch
QBFRELAY								✓		✓	✓	F. Lonsing, U. Egly
QELL								✓		✓	✓	K.H. Tu, T.C. Hsu, J.H.R. Jiang
QESTO	✓		✓				✓	✓	✓	✓	✓	M. Janota
QFUN								✓	✓	✓	✓	M. Janota
QSTS	✓	✓	✓				✓	✓		✓	✓	B. Bogaerts, T. Janhunnen, S. Tasharrofi, J. Devriendt (BID version)
QUABS		✓							✓			L. Tentrup
QUTE								✓	✓	✓	✓	T. Peitl
RAREQS	✓	✓	✓				✓	✓		✓	✓	M. Janota
STRUQS*	✓		✓				✓					L. Pulina, A. Tacchella

tracks: 1... PCNF, 2... PNCNF, 3... 2QBF, 4... EC, 5... SP, 6... PS, 7... RQBF

2QBF formulas This dataset was selected such that it does not overlap with the PCNF set. It is composed of 384 instances in total. It includes the 118 formulas of the 2QBF track of QBFEVAL'16 having hardness coefficient in the range [0, 0.5), the 13 hard instances of the 2QBF track of QBFEVAL'10, a random selection of 12 2QBF instances of hard formulas from QBFEVAL'08, 166 formulas from QBFLIB that were never involved in previous QBFEVALs, as well as a selection of 75 newly submitted formulas.

Random formulas The dataset for the Random track is composed of 505 instances. Of those 319 formulas were selected from the Random track of QBFEVAL'16 with a hardness coefficient in the range [0, 0.5) and the rest was selected from suitable newly submitted sets, including 30 instances of Combinational Equivalence; 113 instances of QBF benchmark for Positional Games; 43 instances of the benchmark set Hard 2QBFs.

2.3. Participating systems

Table 3 summarizes the systems participating in QBFEVAL'16 and/or QBFEVAL'17. Each team of developers was allowed to submit up to three systems per track. As consequences, some tools participated in up to two three versions as described below. Systems that were submitted by an organizer participated hors concours (indicated by a star in Table 3).

AIGSOLVE [17,18] uses And-Inverter Graphs (AIGs) as the main data structure, as well as AIG-based operations to reason about the input formula. Quantifiers are eliminated starting with the inner-most quantifier. The solver includes a dedicated preprocessor [19] based on incremental SAT.

AQUA is a search-based QBF solver using lazy data structures [20] for unit, pure and don't care literal detection. For backtracking, a conflict and solution driven constraint learning approach [21] is used. A restart strategy [22] and phase

saving [23] are also implemented. For QBFEVAL'16, the three versions AQUA-F3V (common first UIP (F-UIP) [24] learning, 3 literals watching, and VSIDS decision heuristic [25]), AQUA-S2V (first semantic UIP (S-UIP) learning, 2 literals watching, and VSIDS), and AQUA-S3O (S-UIP learning, 3 literals watching, and OCCS decision heuristic [25]) have been submitted. Further, AQUA is coupled to the QBF preprocessor SQUEEZE_{BF} [26], which is given a timeout of 100 seconds.

AQME [27] is a multi-engine solver, i.e., a tool using machine learning techniques to select among its reasoning engines the one which is more likely to yield optimal results. The reasoning engines of AQME are a subset of those submitted to QBFEVAL'06. The engine selection is performed according to the adaptive strategy described in [27]. A prototype version coupled to the preprocessor SQUEEZE_{BF} called SQUEEZE_{BF}+AQME has been submitted as well.

AREQS is an implementation of the 2QBF algorithm described in [28] that solves propositional abstractions of the input QBF by using SAT solvers. See also the description of RAREQS.

ASPO is a QBF solver based on ASP solvers that was submitted to QBFEVAL'16. The input formula is preprocessed with BLOQER [29], and then transformed in a ground ASP program according to the classical Eiter-Gottlob encoding of 2QBF in ASP [30], so that it can be evaluated by using an ASP solver. An updated version (ASPO2) has been submitted to QBFEVAL'17.

CADET [31] is a solver for 2QBF formulas based on the incremental construction of the Skolem functions aimed to prove the satisfiability of the formula. An updated version (CADET_2017) has been submitted to QBFEVAL'17.

CAQE [32] is a CEGAR-based approach for QBF. The solver builds upon a decomposition of QBFs into a sequence of propositional formulas that contain the variables of just one quantifier level and additional variables describing the interaction with adjacent quantifier levels. In 2016, two versions of CAQE have been submitted, namely CAQE-MINISAT and CAQE-PICOSAT. Both versions use BLOQER as preprocessor. Additionally, CAQE has been submitted also in non-competitive tracks, in particular CAQE-CERT, CAQE-PORTFOLIO, CAQE-MINISAT-PAR are different solving tools based on the solver CAQE. Two versions of CAQE participated in the EC track, namely CAQE-MINISAT-CERT and CAQE-PICOSAT-CERT. A portfolio variant called CAQE-PORTFOLIO was submitted to the SP track, and two variants participated in the parallel track (CAQE-MINISAT-PAR, CAQE-PICOSAT-PAR). In 2017, three updated versions of CAQE have been submitted, namely CAQE_2017_V1, CAQE_2017_V2, and CAQE_2017_V3.

CHEQ [33] is a suite for QBF certification. It consists of QuBE-CERT—an extension of QuBE_{3.1} able to output certificates—and CHECKER, a tool aimed at checking QuBE-CERT output.

CUED is a QBF solver that is based on Skolem function construction. This solver is incomplete for formulas with more than two quantifier blocks.

DEPQBF [34] is a search-based solver with conflict-driven clause and solution-driven cube learning (QCDCL) [21,35,36]. The variants of DEPQBF submitted to QBFEVAL'16 (DEPQBF-V1, DEPQBF-V2, and DEPQBF-V3) are based on version 5.0 [37], with an advanced technique for early cube learning. The most recent version 6.02 [38] that implements advanced techniques for cube learning participated in QBFEVAL'17, where have been submitted the versions NO-PREFIX-OPT-DEPQBF and PREFIX-OPT-DEPQBF. Further, it also exploits dependency schemes to safely shift variables in the quantifier prefix.

DEPQBF-CERT Two versions of DEPQBF have been submitted for the EC track, namely DEPQBF-CERT-V1, DEPQBF-CERT-V2. The former is intended for the certification of unsatisfiable QBFs, while the latter can certify both satisfiable and unsatisfiable QBFs. Both versions leverage on the QBF_{CERT} [39] framework.

DYNQBF [40] is a structure-aware QBF solver. It splits the QBF instance into sub-problems by constructing a tree decomposition. The QBF is then solved by dynamic programming over the tree decomposition. Intermediate results are stored in sets of binary decision diagrams. DYNQBF used BLOQER and HQSPRE as preprocessors. Further DepQBF 5.01 is used for computing standard dependency schemes. In 2016, one version of it participated to the 2QBF track, while in 2017 three versions have been submitted, namely DYNQBF-BLOQER-HQSPRE, DYNQBF-BLOQER-HQSPRE-IT, DYNQBF-BLOQER-VARIANT. The former participated to the PCNF, 2QBF, and RQBF tracks, while the latter to the 2QBF track only.

GHOSTQ [41] is a non-prenex non-CNF solver that employs duality aware reasoning based on ghost literals. Additionally, it features a counterexample guided abstraction refinement (CEGAR) based learning to further prune the search space when the last decision literal is existential (resp. universal) and a conflict (resp. solution) is detected. Two versions of GHOSTQ have been submitted to QBFEVAL'16, namely GHOSTQ-CEGAR and GHOSTQ-PLAIN. Both participated in the PCNF, 2QBF, and PNCNF tracks. In QBFEVAL'17, the same versions participated in the PNCNF track, but in the CNF tracks, preprocessing was used in addition (versions GHOSTQ-PG_CEGAR and GHOSTQ-PG_PLAIN).

HERETIQ is a hybrid QBF solver that combines QCDCL and expansion-based solving in two separate phases for exploiting the complementary strengths of both QBF solving paradigms. The core solver is based on ITHAD which communicates through learned clauses with the QCDCL solver DepQBF using DepQBF's incremental interface.

HIQER As reported in [6], the QBF solver HIQER consists of a `csh` script that invokes two preprocessors, PLODDE and EQXBF, before passing the resulting QBF to the complete solver STEPQBF. Three versions have been submitted to QBFEVAL'16, namely HIQER1, HIQER3, and HIQER1LDSQ. The parallel version (HIQERFORK) that runs hiqer in different configurations has participated in the PS track.

HORDEQBF [42] is an MPI-based parallel portfolio solver with clause and cube sharing. It is based on the framework of HORDESAT [43], a modular and massively parallel SAT solver. The authors integrated DEPQBF 5.0 in HORDESAT to obtain HORDEQBF.

HQSPRE_SOLVER [44] uses the preprocessor HQSPRE to solve QBFs by performing preprocessing and universal expansion in alternation.

IJTIHAD [45] is an expansion-based QBF solver which generalizes the CEGIS approach for solving QBFs. In contrast to other expansion-based approaches that rely on incremental SAT solvers, **IJTIHAD** uses only two instances of SAT solvers, one that incrementally tries to falsify partial expansions of the universally quantified variables, and one that incrementally tries to show the validity of partial expansions of the existential variables.

IProver [46] is a general purpose theorem prover for first-order logic based on the instantiation calculus **Inst-Gen**. It incorporates a QBF solving mode which is based on a translation of QBF into the effectively propositional fragment of first-order logic (EPR). The basic translation follows the approach of Seidl et al. [47]. Further, it also implements a dedicated Skolemization procedure with several optimization. Two versions of **IProver** have been submitted, namely **IProver-QBF** and **IProver-QBF-BloqQer** (with **BloqQer** for preprocessing). Updated versions have been submitted in 2017.

MPIDepQBF [48] dynamically creates budgeted subproblems by setting outermost variables. The subproblems are solved using **DepQBF**'s implementation of assumptions. The subproblems are then distributed to workers via MPI. The workers use **DepQBF** to solve the subproblems. If a subproblem cannot be solved within a given timeout, it is further split and distributed to idle workers. The master nodes assemble the results of the workers.

PAR-PD-DepQBF. In this solver, the approach to solve quantified circuits in prenex-normal form relies on running two instances of a QBF solver on a primal and a dual version of the problem encoding in parallel – as described in [49]. **PAR-PD-DepQBF** makes use of preprocessing by means of **BloqQer**, and it uses **DepQBF** as back-end PCNF QBF solver.

QBFRelay is an incomplete solver that runs preprocessors **QxBF**, **BloqQer**, and **HQSPRE** in rounds until a fixpoint is reached. In **QBFEVAL'17** **QBFRelay** was also submitted in combination with the solver **DepQBF** to realize a complete solver (**QBFRelay-Limited-DepQBF**).

QELL [50] is an expansion-based approach that uses leveled SAT solving. In order to control formula growth, learning techniques based on circuit structure reconstruction, complete and incomplete **ALLSAT** learning, clause selection, etc. are applied.

QESTO and **QESTOS** are implementations of the QCNF algorithm presented in [51] that are based on the clause selection solving approach. In 2017 **QESTOS** did not participate, while a version for the PNCNF track has been submitted (**cQESTO**).

QFUN is a non-CNF solver that applies machine-learning for learning shorter winning strategies. **QFUN** is based on the expansion-based solver **RAREQS**. It has also been submitted a version for tracks accepting **QDIMACS1.1** as input format (**REV_QFUN**).

QSTS is based on nested SAT solving and theory transformations. The main tools utilized for translation are:

- **SAT-TO-SAT** [52], a (non-)prenex (non-)CNF QBF solver that is based on nested SAT solving, and it is able to do early propagation of information between nested solvers.
- **QBF2STS** [53], a translator from **QDIMACS/QCIR** input format to **SAT-TO-SAT** input format with the ability to reverse engineer circuits and apply several theory transformations to simplify the representation of QBF formulas.

Three versions of **QSTS** have been submitted to **QBFEVAL'16**, namely the plain version (**QSTS**), one version using both **QxBF** [54] and **BloqQer** preprocessors (**XB-QSTS**), and **XB-BID-QSTS**, that extends **XB-QSTS** with **BREAKID** [55], a SAT symmetry breaker that has been modified to detect a (limited) class of symmetries in QBF instances. Updated versions have been submitted in 2017.

QUABS [56] is a certifying QBF solver based on a CEGAR-based abstraction algorithm for Prenex non-CNF formulas in **QCIR** format. Two different versions have been submitted, namely **QUABS-MINISAT** and **QUABS-PICOSAT**, using the SAT solvers **MINISAT** and **PICOSAT**, respectively. An updated version participated at **QBFEVAL'17**.

QUTE [57] is a search-based solver implementing **QCDCL**. It further includes a dedicated technique for the lazy relaxation of the prefix variable ordering called dependency learning.

RAREQS [58] is an expansion-based solver that recursively refines the propositional abstraction of the given QBF. The submitted version uses **BloqQer** as preprocessor. The version **RAREQS-NN** [59] is able to process non-prenex non-CNF formulas in the **QCIR** format.

STRUQS [60], a QBF solver that implements a dynamic combination of search-based solving with solution- and conflict-backjumping and variable-elimination. The key point in this approach is to implicitly leverage graph abstractions of QBFs to yield structural features which support an effective decision between search and variable elimination. A version coupled to the preprocessor **SQUEEZE**BF (**SQUEEZE**BF+**STRUQS**) was submitted as well.

3. Results of QBFEVAL'16

In this section, we summarize the results of **QBFEVAL'16**. With the exception of the solvers submitted to the PS track, all systems were executed as a single process (in some cases as a sequence of several tools like preprocessors and solvers). The CPU time limit was set to 600 seconds and the memory limit was set to 4 GB. All tracks except PS ran on the **StarExec** cluster. The PS track was executed on a cluster of Dell Workstations with Dual Intel Xeon E3-1245 PCs at 3.30 GHz quad core processor, equipped with 64 bit Ubuntu 12.04.

Table 4

Results of QBFEVAL'16 competitive tracks. For each track, we report the number of instances solved within the time limit (“#”) and the total CPU time (in seconds) spent on the solved instances (“Time”). Results in boldface are those of the best three solvers in each track; the colors (gold, silver, bronze) highlight the order. “NA” means that the results are not available (they did not participate in a track or their output gave rise to discrepancies). Finally, solvers marked with a “*” participated *hors-concours*. (For interpretation of the colors in the table(s), the reader is referred to the web version of this article.)

Solver	PCNF		PNCNF		2QBF		RQBF	
	#	Time	#	Time	#	Time	#	Time
AIGSOLVE	589	15981.35	NA	NA	NA	NA	NA	NA
AQUA-F3V	482	7947.80	NA	NA	NA	NA	306	11419.66
AQUA-S3O	479	6774.68	NA	NA	NA	NA	300	10360.07
AQUA-S2V	484	7869.78	NA	NA	NA	NA	306	10976.23
AREQS	NA	NA	NA	NA	235	2963.33	NA	NA
ASPO*	NA	NA	NA	NA	188	741.09	NA	NA
CADET	NA	NA	NA	NA	169	790.78	NA	NA
CAQE-MINISAT	576	15219.10	NA	NA	NA	NA	212	7360.30
CAQE-PICOSAT	590	17178.79	NA	NA	NA	NA	298	7324.49
DEPQBF-V1	456	9999.76	NA	NA	133	5466.70	257	7572.00
DEPQBF-V2	604	14076.91	NA	NA	223	5135.23	287	8977.03
DEPQBF-V3	527	16186.70	NA	NA	138	4901.65	257	7772.75
DYNQBF	NA	NA	NA	NA	72	489.44	NA	NA
GHOSTQ-CEGAR	585	14538.77	524	9009.13	155	8135.25	NA	NA
GHOSTQ-PLAIN	568	13727.80	521	7739.63	87	7545.74	NA	NA
HIQQUER1	NA	NA	NA	NA	183	2703.59	267	6712.26
HIQQUER1LDSQ*	574	10951.54	NA	NA	183	2663.74	267	7013.05
HIQQUER3	NA	NA	NA	NA	185	3236.84	261	4763.45
IProver-QBF	348	12922.04	NA	NA	32	1249.98	25	4341.95
IProver-QBF-BLOQQUER	324	9369.12	NA	NA	124	188.14	59	264.61
QESTO	582	15552.84	NA	NA	NA	NA	291	9398.91
QESTOS	527	4356.04	NA	NA	184	3487.24	246	6904.90
QSTS	NA	NA	NA	NA	NA	NA	239	5231.98
QUABS-MINISAT	NA	NA	503	4287.17	NA	NA	NA	NA
QUABS-PICOSAT	NA	NA	509	4784.62	NA	NA	NA	NA
RAREQS	640	14166.77	NA	NA	232	5287.58	295	4305.78
RAREQS-NN	NA	NA	403	7427.47	NA	NA	NA	NA
SQUEEZEBF+STRUQS*	NA	NA	NA	NA	100	1169.84	41	5851.10
STRUQS*	358	12825.17	NA	NA	100	933.77	35	3623.79
XB-BID-QSTS	NA	NA	NA	NA	NA	NA	NA	NA
XB-QSTS	613	15296.69	NA	NA	206	5581.42	294	7963.03

3.1. Competitive tracks

Table 4 gives an overview of the total results, considering all solvers participating on competitive tracks PCNF, PNCNF, 2QBF, and RQBF. In the PCNF track the solver RAREQS was ranked first, followed by XB-QSTS and DEPQBF-V2. All solvers but STRUQS, IProver-QBF, and IProver-QBF-BLOQQUER were able to solve at least 50% of the dataset. Only RAREQS could solve more than 75% of the instances. Five solvers (out of 19) were able to solve instances uniquely, namely RAREQS (5), AIGSOLVE (24), HIQQUER1LDSQ(1), GHOSTQ-PLAIN, and DEPQBF-V3 (2). For the PCNF track, we excluded five solvers, namely HIQQUER1 (1), HIQQUER3 (1), QSTS (66), SQUEEZEBF+STRUQS (31), and XB-BID-QSTS (18) are not included in the ranking, because they returned at least one discrepant result. See <http://www.qbfeval.org> for details. In order to evaluate discrepant results, we considered both the result “by construction” (if available) of the instance and/or the result returned by the certifying tools submitted to the EC track.

For a more fine-grained view on the results, we introduce the notion of *dominance* as follows: we say that solver *A* dominates solver *B* whenever the set of instances solved by *A* is a superset of the problems solved by *B*. When analyzing the result, it turns out that no solver is dominated by another solver, even when considering RAREQS—the solver ranked first—and IProver-QBF-BLOQQUER—the last solver in the rank. Although IProver-QBF-BLOQQUER is able to solve only about 51% of the total amount of formulas also solved by RAREQS, it turns out that IProver-QBF-BLOQQUER solves six formulas in which RAREQS fails. On the one hand, this indicates the solidity of the considered dataset; on the other hand, this confirms that there is not, in principle, a dominant solving technique for every possible problem.

As shown in Table 4, the two versions of GHOSTQ are the best performing systems in the PNCNF track. GHOSTQ-CEGAR and GHOSTQ-PLAIN rank first and second, respectively, and QUABS-PICOSAT ranks third. All solvers but RAREQS-NN were able to solve at least 50% of the dataset. All of the participating solvers were able to solve instances uniquely. Performance of all the three versions of QSTS are not shown in the table because their output gave rise to discrepancies.

Considering the results related to the 2QBF track, Table 4 reveals that the winner is AREQS, closely followed by RAREQS. The solver DEPQBF-V2 ranks third. AREQS was able to solve the 77% of the dataset, while the last-ranked solver returned

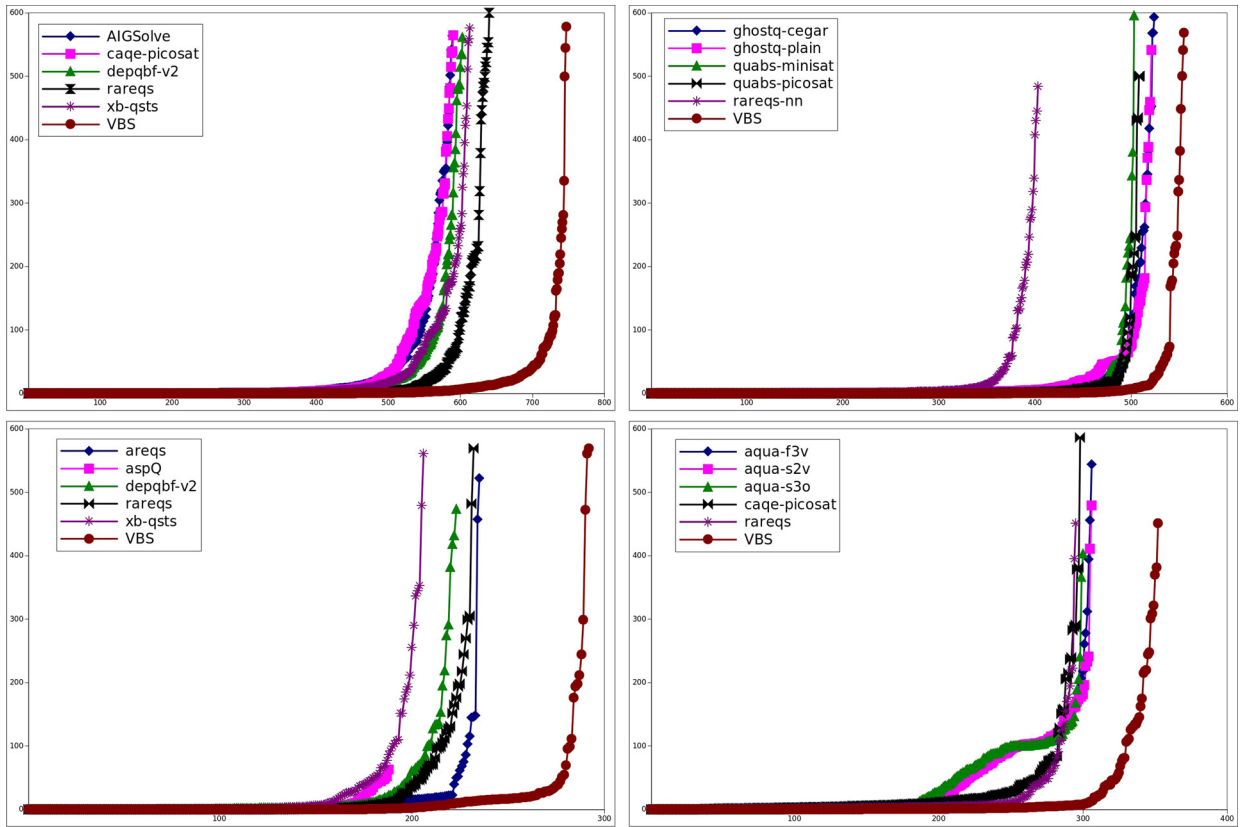


Fig. 1. Performance of top five ranked solvers with respect to the Virtual Best Solver (VBS) on QBFEVAL'16 competitive tracks. Plots in the top show performance of solvers participating at the PCNF and PNCNF Tracks (left and right, respectively). Bottom-left plot is related to the 2QBF Track, while the bottom-right one to the RQBF Track. For each plot, in the x-axes is depicted the total amount of instance, while in the y-axes the CPU time in seconds.

results only for 10% of the dataset. Besides AREQS, only RAREQS solved more than the 75% of the dataset, while 11 solvers (out of 19) could solve 50% of the formulas. Four solvers were able to solve instances uniquely, namely AREQS (1), RAREQS (9), XB-QSTS (1), and CADET (1). We discarded the results of QSTS and XB-BID-QSTS because of discrepancies (one in the case of the former, 15 in the case of the latter). Interestingly, only the six solvers with the fewest solved formulas are dominated by other solvers, while the others are not.

Finally, Table 4 indicates that AQUA is the winner of the RQBF track, where AQUA-S2V, AQUA-F3V, and AQUA-S3O rank first, second, and third, respectively. Seven solvers solved at least 50% of the dataset, while four solvers were not able to cope with more than 10% of the instances. Eight solvers were able to solve formulas uniquely, while for only one of them—XB-BID-QSTS—we have to report discrepancies (16) on the results. About dominance, the four solvers at the bottom of the rank are dominated. Not surprisingly, HIQQR1 and HIQQR1LDSQ dominate HIQQR3, while DEPBQBF-V1 is dominated by AQUA-F3V. All remaining solvers are not dominated.

Fig. 1 depicts the performance of the five top-ranked solvers with respect to the *Virtual Best Solver* (VBS), i.e., the ideal solver that always fares the best time among all the solvers in a portfolio. For each track the portfolio is composed of all participating solvers with the exception of the solvers for which discrepancies have been reported or which are participating *hors-concours*.

In the PCNF track (top-left plot in Fig. 1), the VBS solved 746 instances in 8804.53 seconds; it solved 90% of the dataset, while the winner of the track—RAREQS—successfully solve 77% of the formulas. Out of 746 instances, 367 are true, while 379 are false. Further, 156 formulas can be classified as easy (i.e., solved by all solvers), 34 as medium-hard (i.e., solved by only one solver), and 556 medium. It is worth to notice that 17 solvers contributed to the VBS; in particular, seven solvers contributed for more than 5%, namely DEPBQBF-V1 and DEPBQBF-V3 (8% and 6%, respectively), GHOSTQ-PLAIN and AQUA-F3V (about 9% each), RAREQS (12%), QESTO (22%), and AIGSOLVE (23%). In QBFEVAL'16, special awards were given for distinguished contribution to the VBS. The award was given to AIGSOLVE and QESTO in the PCNF track.

In the PNCNF track (top-right plot in Fig. 1), the VBS solved 555 instances in 6017.16 seconds; it solved the 62% of the dataset, while the winner of the track—GHOSTQ-CEGAR—solved 59%. Out of 555 solved instances, 243 were true, while 312 were false. Overall 375 formulas were turned out to be easy, 20 were medium-hard, and 160 were medium. All five solvers participating in the track contributed to the VBS. The best contributor was QUABS-MINISAT with 69%, while GHOSTQ-CEGAR contributed for about 8%.

Table 5

Results of QBFEVAL'16 non competitive tracks. The table is organized in three groups, one for each track. For each group, we report the name of the system ("System"), the total amount of formulas solved within the time limit ("#") and the total CPU time (in seconds) spent on the solved instances ("Time").

System	EC		System	SP		System	PS	
	#	Time		#	Time		#	Time
DEPQBF-CERT-V2	309	4732.51	CAQE-PORTFOLIO	580	8824.50	PAR-PD-DEPQBF	606	12269.14
CAQE-PICOSAT-CERT	268	7598.09	AQME*	530	9657.69	HIQGERFORK	598	14624.01
CAQE-MINISAT-CERT	236	6831.53	SQUEEZEBF+AQME*	473	9599.09	CAQE-PICOSAT-PAR	585	13337.43
DEPQBF-CERT-V1	217	2760.83				CAQE-MINISAT-PAR	570	12304.04
CHEQ*	217	6188.85				HORDEQBF	443	8434.86

The bottom-left plot in Fig. 1 summarizes the 2QBF track. The VBS solved the 96% of the dataset (292 instances), 57 more than AREQS that won the track and that was also the major contributor (50%) on the performance of the VBS. In total, 205 solved by the VBS were true, while 87 were false, resulting in 13 easy, 14 medium-hard, and 265 medium formulas. Four solvers did not contribute to the VBS, namely DEPQBF-V3, DYNQBF, GHOSTQ-PLAIN, and IPROVER.

Finally, in the RQBF track, as depicted in the bottom-right plot of Fig. 1, the VBS solved 352 instances, 35 more than AQUA-F3V, the winner of the track. For solving these formulas the VBS needed 6296.33 seconds. Out of 352 instances, 146 were found to be true, while 206 were false, resulting in 25 easy, 30 medium-hard, and 297 medium formulas. CAQE-MINISAT and the two version of IPROVER did not contribute to the VBS, while RAREQS (22%) has been awarded as the best VBS contributor.

3.2. Non-competitive tracks

In Table 5 we summarize the results of non-competitive tracks, namely EC, SP, and PS. We did not award any prizes for these tracks.

In EC (left-most part of Table 5) DEPQBF-CERT-V2 is the tool that obtains the highest number of certified answers (309 out of 825). DEPQBF-CERT-V2 was also the tool that could certify the highest amount of false formulas (217), while CHEQ was the tool that could certify the highest amount of true formulas (118). All participating systems were able to solve and certify instances uniquely. In order to have the largest possible set of certified instances, we computed the "Virtual Best Certifier" in a similar way as the VBS in the previous subsection. With the VBC, we obtain 387 certified formulas, 142 of which are true and of which 245 are false.

The SP track was organized as a non-competitive track because two out of three participating systems were *hors-concours*. CAQE-PORTFOLIO was the system that solved the highest amount of formulas, i.e. 580, 10 less than the best version of CAQE submitted to the PCNF track (CAQE-PICOSAT solved 590 instances).

Finally, regarding the PS Track, the best performing system was PAR-PD-DEPQBF, followed by HIQGERFORK and CAQE-PICOSAT-PAR, solving 606, 598, and 585 instances, respectively. It is worth to notice that PAR-PD-DEPQBF solved 3 instances more than the best version of DEPQBF submitted to the PCNF track, while the parallel versions of CAQESolve less formulas than the sequential versions.

4. Results of QBFEVAL'17

In this section, we summarize the results of QBFEVAL'17. The CPU time limit was set to 900 seconds and the memory limit was set to 32 GB. All tracks ran on the StarExec cluster.

Table 6 gives an overview of the QBFEVAL'17 results. In the PCNF track two versions of CAQE are ranked first and second, namely CAQE_2017_V2 and CAQE_2017_V3, and the solver QUTE_RANDOM is ranked third. Only the solvers in the first two positions of the ranking were able to solve more than 50% of the formulas, while IPROVER-QBF-2017 solved less than 25%. The two versions of CUED solved only 7 and 8 instances because, despite they support solving on 2QBF only, they participated also on this track. Eight solvers (out of 30) were able to solve instances uniquely. The solver IJTIHAD_V2 dominates IJTIHAD_V1 and XB-QSTS_XBQSTS2.0 dominates XB-QSTS_XBQSTS1.0. Despite its low performance on the PCNF track, CUED is dominated only by the three versions of CAQE and RAREQS.

As shown in Table 6, QFUN and CQUESTO rank first and second, respectively, in the PNCNF track, while QUABS_2017 ranks third. No solver was able to solve at least 50% of the dataset—QFUN reaches 37%. Four solvers out of eight were able to solve instances uniquely and no solver was dominated.

Regarding the results related to the 2QBF track, Table 6 shows that the winner of the track is GHOSTQ-PG_CEGAR, followed by CADET_2017. CAQE_2017_V2 ranks third. Here the system with the best performance was able to solve the 64% of the formulas, while the last-ranked solver—IPROVER-QBF-2017—solved about 4%. Overall, 17 solvers (out of 29) solved more than 50% of the dataset, while five solvers solved less than 25%. In this track, six solvers were able to solve instances uniquely. The solver XB-QSTS_XBQSTS2.0 dominates XB-QSTS_XBQSTS1.0. In addition, on this dataset CAQE_2017_V2 dominates CAQE_2017_V1 and IPROVER-QBF-2017 is dominated by 18 solvers.

Finally, in the RQBF track, QBFRELAY-LIMITED-DEPQBF is the winner, while RAREQS and CAQE_2017_V2 are ranked second, and third, respectively (cf. Table 6). The dataset of this track was quite hard for the submitted systems; the winner was able

Table 6

Results of QBFEVAL'17 tracks. The table is organized similarly to Table 4. For each track, we report the number of instances solved within the time limit (“#”) and the total CPU time (in seconds) spent on the solved instances (“Time”). Results in boldface are those of the best three solvers in each track; the colors (gold, silver, bronze) high-light the order. “NA” means that the solver did not participate in the related track. Finally, a “–” means that a solver did not solve any instance in the related group.

Solver	PCNF		PNCNF		2QBF		RQBF	
	#	Time	#	Time	#	Time	#	Time
AIGSOLVE	246	18098.45	NA	NA	NA	NA	19	2489.14
ASPQ2	NA	NA	NA	NA	228	8996.81	NA	NA
CADET_2017	NA	NA	NA	NA	241	9276.17	NA	NA
CAQE_2017_V1	230	18773.73	NA	NA	228	15303.55	66	8776.50
CAQE_2017_V2	286	20825.18	NA	NA	230	11755.12	67	6684.96
CAQE_2017_V3	271	19935.09	NA	NA	230	11766.28	66	8244.28
CQESTO	NA	NA	112	8574.74	NA	NA	NA	NA
CUED1919_NL	8	1342.56	NA	NA	73	9493.36	–	–
CUED1919_NNL	7	547.98	NA	NA	70	8972.15	–	–
DYNQBF-BLOQQUER-HQSPRE	223	15335.98	NA	NA	210	7097.44	8	25.50
DYNQBF-BLOQQUER-HQSPRE-IT	NA	NA	NA	NA	207	7265.30	NA	NA
DYNQBF-BLOQQUER-VARIANT	NA	NA	NA	NA	200	5166.79	NA	NA
GHOSTQ-CEGAR	156	11634.73	89	13737.38	76	9162.22	32	4144.25
GHOSTQ-PLAIN	NA	NA	42	6027.87	NA	NA	NA	NA
GHOSTQ-PG_CEGAR	190	16914.39	NA	NA	246	10736.04	41	5611.56
GHOSTQ-PG_PLAIN	163	13512.42	NA	NA	185	8239.49	11	1689.30
HERETIQ	232	18827.34	NA	NA	192	5618.61	58	69668.48
HQSPRE_SOLVER	205	13432.33	NA	NA	NA	NA	9	5.62
IJTIHAD_V1	205	11679.88	NA	NA	161	4344.53	31	1461.48
IJTIHAD_V2	207	11661.28	NA	NA	164	5557.63	33	3851.94
IPROVER-QBF-2017	108	14635.17	NA	NA	16	536.12	4	165.02
IPROVER-QBF-BLOQQUER-2017	150	14442.74	NA	NA	125	584.31	11	933.02
NO-PREFIX-OPT-DEPQBF	NA	NA	NA	NA	74	8030.01	NA	NA
PREFIX-OPT-DEPQBF	157	11596.86	NA	NA			54	5187.86
QBFRELAY	224	14894.21	NA	NA	198	2362.54	14	744.65
QBFRELAY-LIMITED-DEPQBF	236	18342.40	NA	NA	207	18477.13	82	13968.71
QELL_DEFAULT	191	12624.17	NA	NA	103	8484.74	53	5692.99
QELL_UNIT	191	11240.31	NA	NA	104	9846.45	53	5423.12
QESTO	136	11306.00	NA	NA	208	10340.04	58	13790.20
QFUN	NA	NA	117	10607.37	NA	NA	NA	NA
QUABS_2017	NA	NA	106	9105.55	NA	NA	NA	NA
QUTE_DEFAULT	246	19086.20	NA	NA	NA	NA	49	2111.22
QUTE_HYBRID	NA	NA	95	14226.01	NA	NA	NA	NA
QUTE_OPT500	249	21245.50	NA	NA	NA	NA	51	2661.01
QUTE_OPT617	NA	NA	81	7559.50	NA	NA	NA	NA
QUTE_OPT993	NA	NA	86	7005.88	NA	NA	NA	NA
QUTE_RANDOM	250	22330.05	NA	NA	NA	NA	51	2670.65
RAREQS	245	19499.77	NA	NA	229	10833.10	74	7782.52
REV_QFUN	236	19375.71	NA	NA	176	5484.06	33	1737.27
XB-QSTS_BQSTS2.0	188	15097.50	NA	NA	209	10291.68	47	10865.21
XB-QSTS_XBQSTS1.0	191	17608.96	NA	NA	208	11007.97	48	11071.49
XB-QSTS_XBQSTS2.0	191	17602.46	NA	NA	208	10996.27	48	11073.47

to solve about 16% of the instances. Seven solvers (out of 30) were able to solve formulas uniquely and the two versions of CUED did not solve any instance.

Fig. 2 depicts the performance of top five ranked solvers with respect to the VBS. In the PCNF track (top-left plot in Fig. 2), the VBS solved 417 instances in 15,160.95 seconds; it solved about 90% of the dataset, while the winner of the track—CAQE_2017_V2—reached only 55%. Out of 417 instances, 157 have been found to be true, while 260 have been found to be false, overall resulting in 2 easy, 25 medium-hard, and 390 medium formulas. Eight solvers contributed for more than 5%. The major contributors to the VBS were RAREQS and QELL_DEFAULT, with 17% and 14%, respectively.

Concerning VBS performance in the PNCNF track (top-right plot in Fig. 2), we observed that the VBS would solve 181 instances in 11247.52 seconds; this includes the 57% of the dataset, while the winner of the track—QFUN—reaches 37%. Out of 181 instances, 96 have been found to be true, while 85 were false, resulting in 17 easy, 23 medium-hard, and 141 medium formulas. With the exception of GHOSTQ-CEGAR, all solvers participating in the track contributed to the VBS. The most active contributors were QFUN, QUABS_2017 and CQESTO, with 28%, 21%, and 18%, respectively.

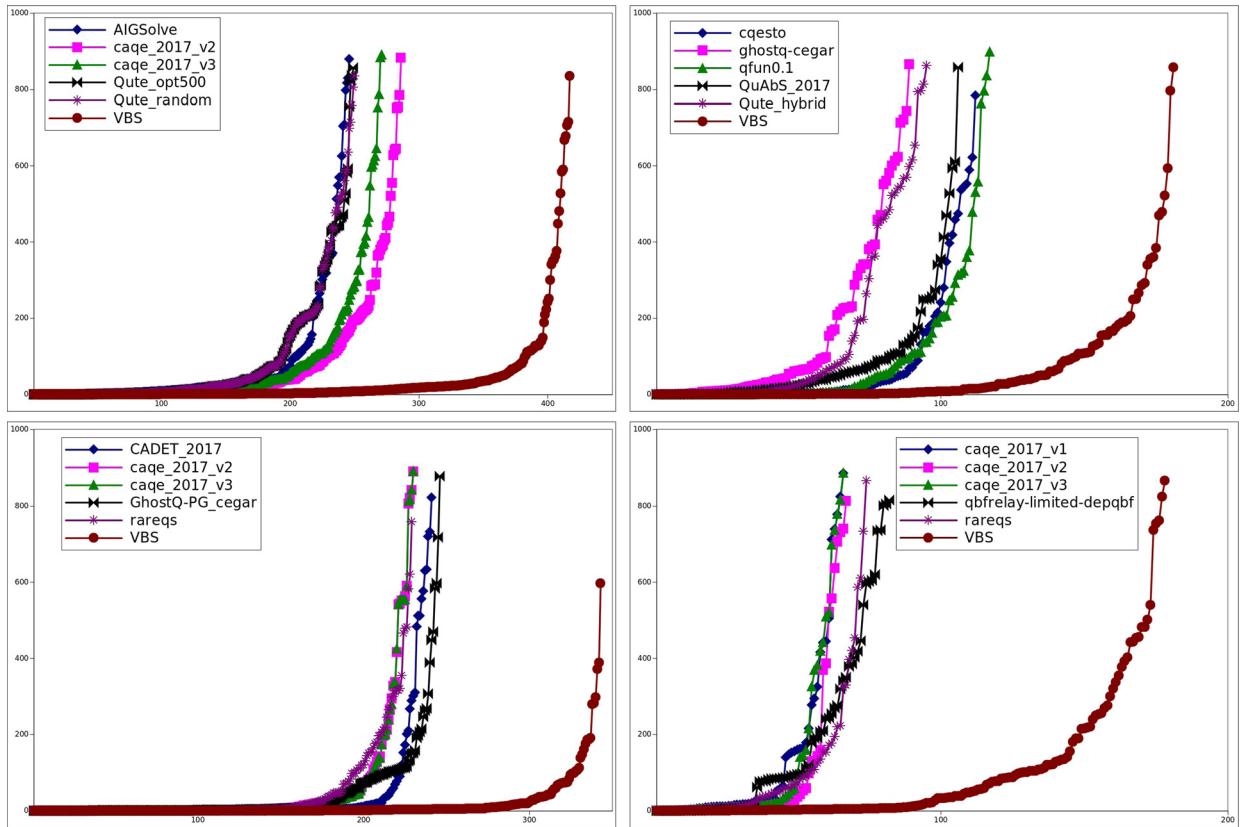


Fig. 2. Performance of top five ranked solvers with respect to the VBS on QBFEVAL'17 tracks. Plots in the top show performance of solvers participating at the PCNF and PNCNF Tracks (left and right, respectively). Bottom-left plot is related to the 2QBF Track, while the bottom-right one to the RQBF Track. For each plot, in the x-axes is depicted the total amount of instance, while in the y-axes the CPU time in seconds.

The bottom-left plot in Fig. 2 shows that the VBS in the 2QBF track solved the 89% of the dataset (343 instances in 5942.92 seconds), 97 more than GHOSTQ-PG_CEGAR. In total, 1212 of the solved formulas were decided to be true, while 131 formulas were false, resulting in 9 easy, 18 medium-hard, and 316 medium formulas. Six solvers contributed to the VBS for more than 5%. The major contributor was CADET_2017 (38%), followed by ASPQ2 (14%).

Finally, the bottom-right plot of Fig. 1 shows that the VBS solved 178 instances in the RQBF track. This is about twice as much as PREFIX-OPT-DEPQBF, the winner of the track, could solve. The CPU time spent by the VBS was 17488.17 seconds. Out of 178 instances, 107 were found to be true and 71 were found to be false, resulting in 28 medium-hard and 150 medium formulas. Seven solvers contributed to the VBS with more than 5%. The major contributor was the winner of the track (20%).

5. Conclusions

In this paper, we reported on the 2016 and 2017 editions of QBFEVAL, the competitive evaluation of solvers for quantified Boolean formulas. As an additional experiment, we took the best three systems of QBFEVAL'16 and run them on the benchmarks of QBFEVAL'17. For this experiment, we considered the tracks PCNF, PNCNF, 2QBF and RQBF. In all of these tracks, the top-ranked solvers of 2017 could solve more instances than the top-ranked solvers of 2016. Considering the benchmark set of the PCNF track in QBFEVAL'17, the top three ranked solvers of QBFEVAL'16, namely DEPQBF-v2, RAREQS, and XB-QSTs, were able to solve 165, 245, and 189 instances, respectively. The winning solver of 2017, CAQE_2017_V3, solved 286 formulas. In the PNCNF track of 2016, the winners were GHOSTQ-CEGAR, GHOSTQ-PLAIN, and QUABS-PICOSAT that solved 85, 42, and 95 formula instances of the 2017 benchmark set. The winning solver of 2017, QFUN, solved 117 formulas. In the 2QBF track, AREQS, DEPQBF-v2, and RAREQS of 2016 solved 184, 188, and 229 formulas. The winner of the 2QBF track, GHOSTQ-CEGAR, solved 246 formulas. Finally, in the RQBF track, the three versions of AQUA solved up to 62 benchmarks of the 2017 dataset, QBFRELAY-LIMITED-DEPQBF – the winner of RQBF in 2017 – solved 82 formulas. In all cases, we hence observe progress on the selected benchmark sets.

As a closing remark, we can say that the huge number of participants indicates that at the moment QBF is a very active research field. While QBF research itself is distributed over different communities like automated reasoning, artificial intelligence, formal verification and synthesis, etc., the QBFEVAL event provides a common platform for exchanging ideas, insights, approaches, tools, and formulas as well as a forum for understanding the *status quo* and for identifying open

challenges. These challenges particularly manifest in terms of the benchmark sets used in the competitions, because these become the basis for the evaluation of novel approaches. Hence, the selection of the benchmarks is a very sensitive task. In 2016, we only relied on the classifications into families as suggested by the submitters of the formulas and on results from previous competitions. However, in this approach we are confronted with the problem that by looking at the outcomes, strengths and weaknesses of the different techniques can have an impact of the selection process. In consequence, we are confronted with the danger of introducing a biased set resulting in overturning that might be counterproductive for the introduction of novel techniques. Hence, a different approach was taken in 2017 by analyzing various features of the benchmark sets and make the selection based on the outcome of classification algorithms. In 2017, we also tried to involve the participants in the selection by inviting them to provide a couple of formulas that they consider interesting and that should be included. Only very few participants used this option, and so we decided to not repeat this again and we will base with feature-based selection process in the next editions of QBFEVAL.

For QBF solving, a lot of different tracks that tackle problems of practical relevance would be of interest. However, we realized that the main focus of QBF tool development currently is on the implementation of sequential P(N)CNF solvers. Therefore, dedicated tracks like on certification, portfolio, and parallel solving do currently hardly find enough participants. Therefore, these tracks were canceled or organized as show-case tracks. In particular, certification is very important – on the one hand for the validation of the result, and on the other hand for the extraction of strategies. Different than in SAT, we cannot make the validation of a result mandatory, because this would exclude several solving techniques for which it is currently not known how to do the certification efficiently. Here more research is required first. Further, we also tried to establish a track on DQBF, a generalization of QBF. This is a very young research field with a lot of potential, and we hope that soon there will be enough contributors (solvers and benchmarks) for organizing a DQBF track.

Finally, detailed statistics from the outcomes of both QBFEVAL events can be obtained at <http://www.qbfeval.org> where custom tables can be generated as described in [61]. For the readers convenience, we included a selection in the appendix. (See Tables 7–22.)

Conflict of interest statement

There is no conflict of interest.

Acknowledgements

The authors would like to thank all the participants of QBFEVAL'16 and QBFEVAL'17 for submitting their work and the judges of QBFEVAL'16 and QBFEVAL'17 for their valuable feedback. Last but not least, the authors would like to thank Aaron Stump and the StarExec team.

Appendix A

Table 7

Results of PCNF Track of QBFEVAL'16. For each solver, the table shows the number of instances solved (“#”) and the total CPU time (in seconds) spent to solve them (“Time”). Total number of formulas solved (“Total”) is also split into true, false, and uniquely solved formulas (“True”, “False”, and “Unique”, respectively). Solvers are sorted according to the number of instances solved, and, in case of a tie, according to CPU time. A dash means that a solver did not solve any instance in the related group. Finally, systems denoted with a “*” participate *hors-concours*.

Solver	Solved		True		False		Unique	
	#	Time	#	Time	#	Time	#	Time
RAREQS	640	14166.77	309	4598.66	331	9568.11	5	985.85
XB-QSTS	613	15296.69	299	5212.69	314	10084.00	–	–
DEPQBF-V2	603	14076.91	297	6256.31	306	7820.60	–	–
CAQE-PICOSAT	590	17178.79	294	6272.92	296	10905.87	–	–
AIGSOLVE	589	15981.35	293	7833.21	296	8148.14	24	1064.73
GHOSTQ-CEGAR	585	14538.77	298	7739.39	287	6799.38	–	–
QESTO	582	15552.84	285	4394.35	297	11158.49	–	–
CAQE-MINISAT	576	15219.10	292	4878.17	284	10340.93	–	–
HIQGERiLDSQ*	574	10951.54	288	6319.74	286	4631.80	1	16.26
GHOSTQ-PLAIN	568	13727.80	282	7000.20	286	6727.60	2	408.42
QESTOS	527	4356.04	252	1848.00	275	2508.04	–	–
DEPQBF-V3	527	16186.70	261	8995.82	266	7190.88	2	1077.76
AQUA-S2V	484	7869.78	229	3290.43	255	4579.35	–	–
AQUA-F3V	482	7947.80	229	3753.27	253	4194.53	–	–
AQUA-S3O	479	6774.68	225	3036.41	254	3738.27	–	–
DEPQBF-V1	456	9999.76	201	4319.97	255	5679.79	–	–
STRUQS*	358	12825.17	175	4595.09	183	8230.08	–	–
IProver-QBF	348	12922.04	158	5385.42	190	7536.62	–	–
IProver-QBF-BLOQER	324	9369.12	243	3955.44	81	5413.68	–	–

Table 8

Classification of the instances of the PCNF Track of QBFEVAL'16. The table consists of nine columns where for each family of instances we report the name of the family in alphabetical order (column "Family"), the number of instances included in the family, the number of instances solved, the number of such instances found TRUE and the number found FALSE (group "Overall", columns "#", "N", "T", and "F", respectively), the time taken to solve the instances (column "Time"), the number of easy, medium and medium-hard instances (group "Hardness", columns "E", "M", and "H").

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
Abduction	10	9	4	5	13.93	0	9	0
Adder	10	9	6	3	422.82	1	3	5
blackbox-01X-QBF	10	10	0	10	10.19	0	10	0
blackbox_design	10	10	10	0	0.92	0	10	0
Blocks	10	10	3	7	0.41	4	6	0
BMC	10	8	5	3	48.47	2	6	0
bomb	10	6	4	2	589.37	0	5	1
C432	8	8	3	5	3.45	4	4	0
C499	8	8	3	5	13.86	3	3	2
C5315	8	5	3	2	6.44	3	2	0
C6288	8	2	2	0	0.67	0	2	0
C880	8	8	2	6	413.80	0	6	2
Chain	10	10	10	0	0.09	10	0	0
circuits	10	4	4	0	202.15	1	3	0
conformant_planning	10	10	6	4	213.92	2	8	0
Connect4	10	6	0	6	10.41	0	6	0
Counter	10	10	10	0	83.80	5	4	1
Debug	10	2	2	0	825.81	0	0	2
DFlipFlop	10	10	0	10	0.42	0	10	0
dungeon	10	10	0	10	134.59	0	5	5
evader-pursuer-4x4-logarithmic	7	7	7	0	1.67	0	7	0
evader-pursuer-4x4-standard	7	7	7	0	86.56	0	7	0
evader-pursuer-6x6-logarithmic	8	5	0	5	182.57	0	5	0
evader-pursuer-6x6-standard	8	3	0	3	340.64	0	3	0
evader-pursuer-8x8-logarithmic	8	5	0	5	351.28	0	5	0
FPGA_PLB_FIT_FAST	5	5	4	1	0.14	4	1	0
FPGA_PLB_FIT_SLOW	3	3	1	2	0.36	0	3	0
fpu	10	10	0	10	5.41	0	10	0
Generalized-Tic-Tac-Toe	10	10	1	9	271.24	0	10	0
HardwareFixpoint	10	9	2	7	268.58	0	5	4
Impl	10	10	10	0	0.00	10	0	0
incrementer-encoder	10	10	1	9	11.32	3	7	0
irqlkeapcte	10	10	10	0	162.67	0	10	0
ISCAS89	7	7	6	1	1.89	3	4	0
ITC99	7	4	3	1	25.65	1	3	0
jmc_quant_squaring	10	8	4	4	27.66	0	4	4
k_branch_n	10	9	9	0	12.48	3	6	0
k_branch_p	10	9	0	9	26.04	0	9	0
k_d4_n	10	10	10	0	0.76	3	7	0
k_d4_p	10	10	0	10	0.26	0	10	0
k_dum_n	10	10	10	0	0.09	4	6	0
k_dum_p	10	10	0	10	0.09	0	10	0
k_grz_n	10	10	10	0	0.33	7	3	0
k_grz_p	10	10	0	10	0.25	0	10	0
k_lin_n	10	10	10	0	6.24	6	4	0
k_lin_p	10	10	0	10	0.19	0	10	0
k_path_n	10	10	10	0	0.18	4	6	0
k_path_p	10	10	0	10	0.23	0	10	0
k_ph_n	10	10	10	0	23.83	8	2	0
k_ph_p	10	4	0	4	40.02	1	3	0
k_poly_n	10	10	10	0	0.07	1	9	0
k_poly_p	10	10	0	10	0.08	0	10	0
k_t4p_n	10	10	10	0	0.49	1	9	0
k_t4p_p	10	10	0	10	0.38	0	10	0
LinearBitvectorRankingFunction	10	3	1	2	501.02	0	2	1
Logn	4	4	0	4	0.86	1	3	0
mqm	10	10	5	5	158.10	0	9	1
MutexP	7	7	7	0	0.32	1	6	0
Planning-CTE	7	7	2	5	181.77	2	5	0
QBF-Hardness	10	10	1	9	32.15	2	8	0
qbfeval12	6	6	2	4	31.87	4	2	0
Qshifter	6	6	6	0	6.00	3	3	0
RankingFunctions	10	10	10	0	0.31	0	10	0
Reduction-finding	10	9	2	7	43.42	2	6	1
Rewriting	10	10	0	10	0.02	0	10	0

Table 8 (continued)

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
s1196	6	6	1	5	308.64	0	6	0
s1269	10	5	5	0	128.89	0	5	0
s27	4	4	1	3	0.03	1	3	0
s298	10	10	7	3	89.46	1	9	0
s3330	10	5	5	0	405.17	0	5	0
s386	10	10	5	5	36.97	0	10	0
s499	10	10	7	3	183.41	0	10	0
s510	10	10	10	0	707.74	0	10	0
s641	9	9	5	4	105.82	0	9	0
s713	10	10	5	5	203.31	0	10	0
s820	10	10	6	4	215.89	0	10	0
Sorting_networks	10	10	5	5	220.51	1	7	2
SzymanskiP	10	10	0	10	114.76	0	10	0
term1	8	8	4	4	0.72	6	2	0
terminator	10	10	1	9	194.47	0	9	1
tipdiam	10	10	7	3	44.85	4	4	2
tipfixpoint	10	10	7	3	15.39	1	9	0
ToiletA	10	10	3	7	0.37	8	2	0
ToiletC	10	10	1	9	0.69	7	3	0
ToiletG	7	7	7	0	0.00	7	0	0
trafficlight-controller	10	10	0	10	20.79	3	7	0
Tree	10	10	2	8	0.00	1	9	0
uclid	3	2	1	1	2.68	0	2	0
VonNeumann	10	10	0	10	2.79	0	10	0
wmiforward	10	10	10	0	0.17	3	7	0
z4ml	8	8	4	4	0.00	4	4	0

Table 9

Results of PNCNF Track of QBFEVAL'16. The table is organized as Table 7.

Solver	Solved		True		False		Unique	
	#	Time	#	Time	#	Time	#	Time
GHOSTQ-CEGAR	524	9009.13	231	5391.70	293	3617.43	8	1907.06
GHOSTQ-PLAIN	521	7739.63	229	2802.33	292	4937.30	4	1055.21
QUABS-PICOSAT	509	4784.62	223	2047.07	286	2737.55	5	761.05
QUABS-MINISAT	503	4287.17	217	2608.65	286	1678.52	1	248.48
RAREQS-NN	403	7427.47	174	3161.98	229	4265.49	2	208.49

Table 10

Classification of the instances of the PNCNF Track of QBFEVAL'16. The table is organized as Table 8.

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
Abduction	5	4	1	3	0.86	3	1	0
Adder	5	1	0	1	0.10	1	0	0
blackbox-01X-QBF	5	5	0	5	1.03	4	1	0
blackbox_design	5	5	5	0	0.74	5	0	0
Blocks	5	5	2	3	4.31	4	1	0
BMC	5	2	1	1	2.09	2	0	0
bomb	5	2	1	1	20.37	0	2	0
C432	4	4	2	2	5.03	4	0	0
C499	4	2	1	1	0.56	1	1	0
C5315	4	3	1	2	2.40	2	1	0
C6288	4	0	0	0	–	0	0	0
C880	4	3	0	3	859.45	1	1	1
Chain	5	5	5	0	2.94	1	4	0
circuits	5	1	1	0	0.10	1	0	0
conformant_planning	6	4	3	1	187.99	2	2	0
Connect4	5	3	0	3	17.12	2	1	0
Counter	5	3	3	0	40.81	1	2	0
Debug	5	0	0	0	–	0	0	0
DFlipFlop	5	5	0	5	46.30	5	0	0
dungeon	5	2	0	2	14.18	0	2	0
evader-pursuer-4x4-logarithmic	4	4	4	0	29.20	0	3	1
evader-pursuer-4x4-standard	4	0	0	0	–	0	0	0
evader-pursuer-6x6-logarithmic	4	1	0	1	168.79	0	0	1

(continued on next page)

Table 10 (continued)

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
evader-pursuer-6x6-standard	4	0	0	0	–	0	0	0
evader-pursuer-8x8-logarithmic	4	1	0	1	70.35	0	1	0
FPGA_PLB_FIT_FAST	3	3	2	1	4.60	2	1	0
FPGA_PLB_FIT_SLOW	2	2	0	2	12.45	0	2	0
fpu	5	5	0	5	116.21	2	3	0
Generalized-Tic-Tac-Toe	5	5	1	4	53.07	0	5	0
HardwareFixpoint	5	0	0	0	–	0	0	0
Impl	5	5	5	0	0.18	0	5	0
incrementer-encoder	5	5	1	4	243.99	1	1	3
irqlkeapcte	5	4	4	0	1604.25	0	0	4
ISCAS89	3	2	2	0	0.98	0	2	0
ITC99	5	3	3	0	121.82	0	2	1
jmc_quant_squaring	5	0	0	0	–	0	0	0
k_branch_n	5	4	4	0	262.70	1	3	0
k_branch_p	5	5	0	5	87.14	0	4	1
k_d4_n	5	5	5	0	5.91	1	4	0
k_d4_p	5	5	0	5	2.54	0	5	0
k_dum_n	5	5	5	0	0.92	1	4	0
k_dum_p	5	5	0	5	0.57	0	5	0
k_grz_n	5	5	5	0	0.74	0	5	0
k_grz_p	5	5	0	5	1.24	0	5	0
k_lin_n	5	5	5	0	4.43	1	4	0
k_lin_p	5	5	0	5	0.10	5	0	0
k_path_n	5	5	5	0	1.07	0	5	0
k_path_p	5	5	0	5	0.78	1	4	0
k_ph_n	5	5	5	0	7.19	2	2	1
k_ph_p	5	3	0	3	48.28	2	1	0
k_poly_n	5	5	5	0	1.51	0	5	0
k_poly_p	5	5	0	5	1.30	0	5	0
k_t4p_n	5	5	5	0	8.29	0	5	0
k_t4p_p	5	5	0	5	4.69	0	5	0
LinearBitvectorRankingFunction	5	1	0	1	30.25	0	0	1
Logn	2	2	0	2	33.18	2	0	0
mqm	136	135	1	134	74.23	132	3	0
MutexP	4	4	4	0	2.12	1	3	0
NuSMV_diam	92	92	92	0	26.44	92	0	0
Planning-CTE	3	1	1	0	3.75	1	0	0
QBF-Hardness	5	4	1	3	29.9	2	2	0
qbfeval12	2	2	0	2	0.45	0	2	0
QLTL_safety	250	11	0	11	59.93	3	8	0
Qshifter	3	1	1	0	0.00	1	0	0
RankingFunctions	5	0	0	0	–	0	0	0
Reduction-finding	5	3	0	3	3.04	3	0	0
Rewriting	5	5	0	5	0.01	5	0	0
s1196	3	3	1	2	11.65	1	2	0
s1269	5	2	2	0	0.58	2	0	0
s27	2	2	0	2	0.00	2	0	0
s298	5	5	2	3	3.61	2	3	0
s3330	5	3	3	0	1.01	3	0	0
s386	5	5	2	3	0.58	5	0	0
s499	5	5	3	2	2.40	5	0	0
s510	5	5	5	0	8.54	4	1	0
s641	5	5	3	2	8.51	3	2	0
s713	5	5	3	2	11.74	3	2	0
s820	5	5	2	3	2.93	5	0	0
Sorting_networks	5	1	1	0	10.14	0	1	0
SzymanskiP	5	5	0	5	728.38	0	2	3
term1	4	4	2	2	1.05	4	0	0
terminator	5	5	0	5	0.51	3	2	0
tipdiam	5	4	3	1	248.52	3	0	1
tipfixpoint	5	5	4	1	0.49	4	1	0
ToiletA	5	5	0	5	3.15	5	0	0
ToiletC	5	5	1	4	10.59	4	1	0
ToiletG	4	4	4	0	0.02	4	0	0
trafficlight-controller	5	5	0	5	4.60	3	2	0
Tree	5	5	1	4	0.13	1	4	0
uclid	2	1	1	0	336.35	0	0	1
VonNeumann	5	5	0	5	285.76	3	1	1
wmiforward	5	5	5	0	0.95	2	3	0
z4ml	4	4	3	1	0.00	4	0	0

Table 11

Results of 2QBF Track of QBFEVAL'16. The table is organized as Table 7.

Solver	Solved		True		False		Unique	
	#	Time	#	Time	#	Time	#	Time
AREQS	235	2963.33	179	2136.52	56	826.81	1	22.22
RAREQS	232	5287.58	156	2084.94	76	3202.64	9	1801.46
DEPQBF-V2	223	5135.23	142	1553.21	81	3582.02	–	–
XB-QSTS	206	5581.42	154	3354.41	52	2227.01	1	561.03
ASPO*	188	741.09	141	275.41	47	465.68	–	–
HIQQUER3	185	3236.84	150	2235.98	35	1000.86	–	–
QUESTOS	184	3487.24	135	1194.36	49	2292.88	–	–
HIQQUER1LDSQ*	183	2663.74	147	2195.58	36	468.16	–	–
HIQQUER1	183	2703.59	147	2232.26	36	471.33	–	–
CADET	169	790.78	120	512.95	49	277.83	1	472.50
GHOSTQ-CEGAR	155	8135.25	108	6031.48	47	2103.77	–	–
DEPQBF-V3	138	4901.65	97	1799.51	41	3102.14	–	–
DEPQBF-V1	133	5466.70	68	1262.27	65	4204.43	–	–
IProver-QBF-BLOQQUER	124	188.14	122	78.66	2	109.48	–	–
STRUQS*	100	933.77	73	483.18	27	450.59	–	–
SQUEEZEBF+STRUQS*	100	1169.84	73	720.19	27	449.65	–	–
GHOSTQ-PLAIN	87	7545.74	40	4115.46	47	3430.28	–	–
DYNQBF	72	489.44	70	489.29	2	0.15	–	–
IProver	32	1249.98	30	1142.63	2	107.35	–	–

Table 12

Classification of the instances of the 2QBF Track of QBFEVAL'16. The table is organized as Table 8.

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
irqlkeapcte	46	46	46	0	607.53	0	45	1
k_ph_n	1	1	1	0	0.00	1	0	0
MutexP	7	7	7	0	0.34	1	6	0
Qshifter	6	6	6	0	6.01	3	3	0
RankingFunctions	50	50	49	1	1.61	0	50	0
Reduction-finding	48	37	24	13	2267.48	0	26	11
Sorting_networks	42	40	15	25	1286.11	0	39	1
terminator	50	50	2	48	505.67	0	49	1
Tree	5	5	5	0	0.00	3	2	0
wmiforward	50	50	50	0	2.80	5	45	0

Table 13

Results of RQBF Track of QBFEVAL'16. The table is organized as Table 7.

Solver	Solved		True		False		Unique	
	#	Time	#	Time	#	Time	#	Time
AQUA-S2V	306	10976.23	127	4952.36	179	6023.87	2	9.63
AQUA-F3V	306	11419.66	127	5031.04	179	6388.62	1	22.47
AQUA-S3O	300	10360.07	127	5085.79	173	5274.28	–	–
CAQE-PICOSAT	298	7324.49	128	2579.82	170	4744.67	6	869.06
RAREQS	295	4305.78	127	1699.99	168	2605.79	4	762.33
XB-QSTS	294	7963.03	132	2295.17	162	5667.86	4	535.47
QUESTO	291	9398.91	131	3720.64	160	5678.27	1	308.61
DEPQBF-V2	287	8977.03	113	1613.30	174	7363.73	–	–
HIQQUER1	267	6712.26	111	1709.68	156	5002.58	–	–
HIQQUER1LDSQ*	267	7013.05	111	1681.42	156	5331.63	–	–
HIQQUER3	261	4763.45	111	1560.89	150	3202.56	–	–
DEPQBF-V1	257	7572.00	104	2130.39	153	5441.61	–	–
DEPQBF-V3	257	7772.75	108	2726.88	149	5045.87	1	37.04
QUESTOS	246	6904.90	107	2340.14	139	4564.76	–	–
QSTS	239	5231.98	106	2339.49	133	2892.49	10	564.45
CAQE-MINISAT	212	7360.30	107	3758.12	105	3602.18	–	–
IProver-QBF-BLOQQUER	59	264.61	58	75.49	1	189.12	–	–
SQUEEZEBF+STRUQS*	41	5851.10	27	3814.07	14	2037.03	–	–
STRUQS*	35	3623.79	21	1602.65	14	2021.14	–	–
IProver-QBF	25	4341.95	25	4341.95	–	–	–	–

Table 14

Classification of the instances of the RQBF Track of QBFEVAL'16. The table is organized as Table 8.

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
ASP_Program_Inclusion	40	40	0	40	26.63	0	40	0
CounterFactual	80	61	27	34	1163.47	4	47	10
Model_instances	60	40	24	16	735.45	0	38	2
Q_2_2_3	50	0	0	0	–	0	0	0
Q_2_3	50	30	10	20	589.29	0	19	11
Q_3_3	50	1	0	1	3.73	0	0	1
Q_3_3_3	50	0	0	0	–	0	0	0
RobotsD2	30	30	26	4	853.83	3	24	3
RobotsD3	30	30	23	7	116.66	6	24	0
RobotsD4	30	30	15	15	49.35	6	24	0
RobotsD5	30	30	13	17	32.92	6	24	0
Strategic_Companies	80	60	8	52	2725.00	0	57	3

Table 15

Results of PCNF Track of QBFEVAL'17. For each solver, the table shows the number of instances solved (“#”) and the total CPU time (in seconds) spent to solve them (“Time”). Total number of formulas solved (“Total”) is also split into true, false, and uniquely solved formulas (“True”, “False”, and “Unique”, respectively). Solvers are sorted according to the number of instances solved, and, in case of a tie, according to CPU time. A dash means that a solver did not solve any instance in the related group. Finally, systems denoted with a “*” participate *hors-concours*.

Solver	Solved		True		False		Unique	
	#	Time	#	Time	#	Time	#	Time
CAQE_2017_V2	286	20825.18	86	6807.57	200	14017.61	–	–
CAQE_2017_V3	271	19935.09	84	8195.79	187	11739.30	–	–
QUTE_RANDOM	250	22330.05	77	8875.44	173	13454.61	–	–
QUTE_OPT500	249	21245.50	77	8604.45	172	12641.05	–	–
AIGSolve	246	18098.45	78	5685.02	168	12413.43	12	777.62
QUTE_DEFAULT	246	19086.20	67	6350.96	179	12735.24	–	–
RAREQS	245	19499.77	73	5684.27	172	13815.50	1	480.97
QBFRELAY-LIMITED-DEPQBF	236	18342.40	79	6011.29	157	12331.11	1	715.80
REV_QFUN	236	19375.71	89	5723.70	147	13652.01	4	1406.47
HERETIQ	232	18827.34	67	7926.39	165	10900.95	1	835.00
CAQE_2017_V1	230	18773.73	75	5668.16	155	13105.57	–	–
QBFRELAY	224	14894.21	77	3657.70	147	11236.51	–	–
DYNQBF-BLOQQER-HQSPRE	223	15335.98	68	2406.87	155	12929.11	3	470.28
IJTIHAD_V2	207	11661.28	55	2559.76	152	9101.52	–	–
IJTIHAD_V1	205	11679.88	54	2983.89	151	8695.99	–	–
HQSPRE_SOLVER	205	13432.33	63	5406.01	142	8026.32	–	–
QELL_UNIT	191	11240.31	56	4429.34	135	6810.97	–	–
QELL_DEFAULT	191	12624.17	54	4198.90	137	8425.27	–	–
XB-QSTS_XBQSTS2.0	191	17602.46	58	6723.96	133	10878.50	–	–
XB-QSTS_XBQSTS1.0	191	17608.96	58	6724.34	133	10884.62	–	–
GhostQ-PG_cegar	190	16914.39	70	7001.93	120	9912.46	1	25.93
xb-qsts_bqsts2.0	188	15097.50	56	4931.72	132	10165.78	–	–
GHOSTQ-PG_PLAIN	163	13512.42	54	4231.14	109	9281.28	2	929.62
PREFIX-OPT-DEPQBF	157	11596.86	41	2652.72	116	8944.14	–	–
GHOSTQ-CEGAR	156	11634.73	61	5116.05	95	6518.68	–	–
IProver-QBF-BLOQQER-2017	150	14442.74	47	4278.55	103	10164.19	–	–
QESTO	136	11306.00	45	4202.79	91	7103.21	–	–
IProver-QBF-2017	108	14653.17	18	2273.93	90	12379.24	–	–
CUED1919_NL	8	1342.56	4	106.86	4	1235.70	–	–
CUED1919_NNL	7	547.98	4	100.50	3	447.48	–	–

Table 16

Classification of the instances of the PCNF Track of QBFEVAL'17. The table is organized as Table 8.

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
Abduction	5	2	1	1	45.26	0	2	0
Adder	10	10	7	3	94.40	0	8	2
amba	4	3	1	2	136.82	0	3	0
arithmetic	5	5	5	0	9.01	0	5	0
blackbox-01X-QBF	20	20	0	20	30.56	0	20	0
BMC	13	6	4	2	138.43	0	6	0
C432	2	2	0	2	0.28	0	2	0
C499	3	3	0	3	14.37	0	3	0

Table 16 (continued)

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
C5315	5	2	1	1	2.99	0	2	0
C6288	7	3	3	0	733.11	0	1	2
C880	6	6	0	6	568.07	0	5	1
circuits	14	3	3	0	90.21	0	3	0
conformant_planning	10	8	3	5	881.55	0	8	0
Connect2	1	1	1	0	9.56	0	1	0
Connect3	1	0	0	0	-	0	0	0
Connect4	8	1	0	1	0.05	0	1	0
Connect5	2	2	0	2	128.14	0	2	0
Connect6	2	1	0	1	0.05	0	1	0
Connect7	2	0	0	0	-	0	0	0
Connect8	2	2	0	2	0.36	0	2	0
Counter	5	3	3	0	55.25	0	2	1
cycle-sched	4	4	2	2	65.23	0	4	0
Debug	14	7	7	0	1787.95	0	7	0
disjunctive_decomposition	5	4	3	1	1.73	0	4	0
driver	4	4	2	2	0.37	0	4	0
dungeon	26	26	1	25	465.42	0	26	0
evader-pursuer-4x4-logarithmic	1	1	1	0	0.48	0	1	0
evader-pursuer-4x4-standard	7	7	7	0	14.23	0	7	0
evader-pursuer-6x6-logarithmic	5	3	0	3	1013.88	0	2	1
evader-pursuer-6x6-standard	8	2	0	2	187.68	0	2	0
evader-pursuer-8x8-logarithmic	5	3	0	3	1255.86	0	2	1
formula_add	12	9	9	0	1411.26	0	7	2
fpu	20	20	0	20	23.32	0	20	0
genbuf	4	3	1	2	411.32	0	2	1
Generalized-Tic-Tac-Toe	7	7	1	6	144.00	0	7	0
genpatch	5	5	3	2	106.20	0	5	0
HardwareFixpoint	26	17	2	15	568.74	1	16	0
hwmcc	4	4	2	2	141.22	0	4	0
hyperLTL	2	2	1	1	0.00	1	1	0
incrementer-encoder	13	13	2	11	23.31	0	13	0
irqlkeapclte	10	10	10	0	107.39	0	10	0
ISCAS89	4	4	2	2	1.04	0	4	0
ITC99	9	8	7	1	736.08	0	6	2
jmc_quant	1	0	0	0	-	0	0	0
jmc_quant_squaring	8	6	3	3	16.73	0	3	3
k_branch_n	6	6	6	0	133.19	0	6	0
k_branch_p	8	8	0	8	20.19	0	8	0
k_ph_p	10	5	0	5	618.17	0	2	3
LinearBitvectorRankingFunction	23	9	6	3	835.58	0	9	0
ltl2aig-comp	4	2	0	2	21.61	0	2	0
LTL2DBA	2	2	1	1	0.24	0	2	0
LTL2DPA	2	2	1	1	1.50	0	2	0
mqm	3	3	3	0	45.89	0	3	0
mult-matrix	4	4	2	2	41.47	0	2	2
Planning-CTE	29	29	2	27	610.59	0	29	0
QBF-Hardness	10	10	1	9	152.85	0	10	0
qbfeval12	3	3	3	0	14.03	0	3	0
RankingFunctions	2	2	2	0	0.08	0	2	0
Reduction-finding	6	6	3	3	74.36	0	5	1
s1196	2	2	0	2	101.35	0	2	0
s1269	2	1	1	0	3.33	0	1	0
s298	2	2	1	1	9.58	0	2	0
s3330	2	1	1	0	2.92	0	1	0
s499	2	2	1	1	2.35	0	2	0
s510	2	2	2	0	36.64	0	2	0
s641	2	2	1	1	8.70	0	2	0
s713	2	2	0	2	51.37	0	2	0
s820	2	2	1	1	3.36	0	2	0
sketch	5	2	0	2	413.67	0	2	0
Sorting_networks	6	6	2	4	96.05	0	6	0
SzymanskiP	2	2	0	2	34.98	0	2	0
terminator	9	9	1	8	12.46	0	9	0
tipdiam	10	10	5	5	200.18	0	8	2
tipfixpoint	13	12	9	3	157.78	0	11	1
toy	4	4	2	2	0.42	0	4	0
trafficlight-controller	10	10	0	10	10.63	0	10	0
uclid	3	3	2	1	23.52	0	3	0

Table 17

Results of PNCNF Track of QBFEVAL'17. The table is organized as Table 15.

Solver	Solved		True		False		Unique	
	#	Time	#	Time	#	Time	#	Time
QFUN	117	10607.37	56	4485.13	61	6122.24	11	2398.47
CQUESTO	112	8574.74	57	4782.37	55	3792.37	–	–
QUABS_2017	106	9105.55	48	4355.14	58	4750.41	7	1935.29
QUTE_HYBRID	95	14226.01	43	6869.27	52	7356.74	4	893.89
GHOSTQ-CEGAR	89	13737.38	41	7693.15	48	6044.23	–	–
QUTE_OPT993	86	7005.88	40	3289.58	46	3716.30	–	–
QUTE_OPT617	81	7559.50	37	3263.29	44	4296.21	1	156.40
GHOSTQ-PLAIN	42	6027.87	16	2157.24	26	3870.63	–	–

Table 18

Classification of the instances of the PNCNF Track of QBFEVAL'17. The table is organized as Table 8.

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
Abduction	1	0	0	0	–	0	0	0
Adder	4	0	0	0	–	0	0	0
amba	4	2	1	1	400.14	0	2	0
Blocks	1	1	0	1	0.42	0	1	0
BMC	3	1	1	0	106.98	0	1	0
bomb	3	2	1	1	269.39	0	0	2
BoundedSynthesisPetriGames	42	23	10	13	2816.69	5	17	1
C499	3	3	0	3	2.91	0	3	0
C5315	1	0	0	0	–	0	0	0
C6288	4	1	1	0	196.18	0	0	1
C880	2	1	0	1	2.44	0	1	0
Chain	3	3	3	0	0.34	0	3	0
circuits	4	1	1	0	360.12	0	0	1
CombinationalEquivalence	30	30	15	15	17.41	0	30	0
conformant_planning	3	2	2	0	117.61	0	1	1
Connect4	2	0	0	0	–	0	0	0
Counter	2	1	1	0	87.64	0	1	0
cycle-sched	4	1	0	1	796.48	0	0	1
Debug	5	0	0	0	–	0	0	0
driver	4	3	1	2	2.33	1	2	0
dungeon	3	1	0	1	1.56	0	1	0
evader-pursuer-4x4-logarithmic	2	2	2	0	5.37	0	2	0
evader-pursuer-4x4-standard	4	0	0	0	–	0	0	0
evader-pursuer-6x6-logarithmic	4	1	0	1	166.35	0	1	0
evader-pursuer-6x6-standard	4	0	0	0	–	0	0	0
evader-pursuer-8x8-logarithmic	4	1	0	1	19.05	0	1	0
fpu	2	2	0	2	14.25	2	0	0
genbuf	4	2	0	2	482.28	0	2	0
HardwareFixpoint	5	0	0	0	–	0	0	0
hwmcc	4	2	0	2	2.14	2	0	0
hyperLTL	2	2	1	1	0.00	2	0	0
incrementer-encoder	3	3	1	2	0.88	0	3	0
irqlkeapcte	5	5	5	0	180.62	0	5	0
ISCAS89	1	1	1	0	18.46	0	1	0
ITC99	4	4	4	0	225.46	0	2	2
jmc_quant_squaring	5	1	0	1	266.44	0	1	0
k_branch_n	2	2	2	0	114.54	0	0	2
k_branch_p	3	3	0	3	11.72	0	2	1
k_ph_n	1	1	1	0	0.24	0	1	0
k_ph_p	2	1	0	1	31.59	0	0	1
LinearBitvectorRankingFunction	5	1	0	1	0.66	0	1	0
ltl2aig-comp	4	2	0	2	116.28	0	1	1
LTL2DBA	2	1	0	1	0.27	1	0	0
LTL2DPA	2	1	0	1	0.64	0	1	0
Model_instances	20	14	10	4	811.94	0	11	3
mqm	2	2	1	1	469.94	0	1	1
mult-matrix	4	0	0	0	–	0	0	0
MutexP	3	3	3	0	0.12	0	3	0
Planning-CTE	2	1	0	1	478.93	0	0	1
QBF-Hardness	2	1	1	0	6.33	0	1	0
QLTL_safety	50	28	20	8	2142.86	1	24	3
Qshifter	2	0	0	0	–	0	0	0
RankingFunctions	5	0	0	0	–	0	0	0
Reduction-finding	2	0	0	0	–	0	0	0
s1269	3	0	0	0	–	0	0	0
s3330	2	0	0	0	–	0	0	0

Table 18 (continued)

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
Sorting_networks	4	2	1	1	33.16	0	1	1
SzymanskiP	5	5	0	5	41.53	0	5	0
terminator	1	1	0	1	3.37	0	1	0
tipdiam	2	1	0	1	340.32	0	1	0
toy	4	4	2	2	0.51	2	2	0
uclid	2	2	2	0	68.96	0	2	0
VonNeumann	1	1	0	1	13.63	1	0	0
wmiforward	2	2	2	0	0.04	0	2	0

Table 19

Results of 2QBF Track of QBFEVAL'17. The table is organized as Table 15.

Solver	Solved		True		False		Unique	
	#	Time	#	Time	#	Time	#	Time
GHOSTQ-PG_CEGAR	246	10736.04	141	6909.24	105	3826.80	3	1178.17
CADET_2017	241	9276.17	134	7529.46	107	1746.71	5	106.18
CAQE_2017_V2	230	11755.12	117	5122.23	113	6632.89	–	–
CAQE_2017_V3	230	11766.28	117	5130.78	113	6635.50	–	–
RAREQS	229	10833.10	139	5140.59	90	5692.51	1	106.89
ASPQ2	228	8996.81	145	4981.43	83	4015.38	6	948.28
CAQE_2017_V1	228	15303.55	141	8476.27	87	6827.28	–	–
DYNQBF-BLOQER-HQSPRE	210	7097.44	142	2376.55	68	4720.89	–	–
XB-QSTS_BQSTS2.O	209	10291.68	132	7822.38	77	2469.30	–	–
QESTO	208	10340.04	132	6384.94	76	3955.10	–	–
XB-QSTS_XBQSTS2.O	208	10996.27	132	7953.58	76	3042.69	–	–
XB-QSTS_XBQSTS1.O	208	11007.97	132	7958.92	76	3049.05	–	–
DYNQBF-BLOQER-HQSPRE-IT	207	7265.30	139	2248.07	68	5017.23	–	–
QBFRELAY-LIMITED-DEPQBF	207	18477.13	122	11335.91	85	7141.22	–	–
DYNQBF-BLOQER-VARIANT	200	5166.79	134	2219.50	66	2947.29	2	484.36
QBFRELAY	198	2362.54	140	1444.20	58	918.34	–	–
HERETIQ	192	5618.61	103	2496.97	89	3121.64	–	–
GHOSTQ-PG_PLAIN	185	8239.49	79	5025.16	106	3214.33	–	–
REV_QFUN	176	5484.06	122	3386.80	54	2097.26	–	–
IJTIHAD_V2	164	5557.63	99	4250.96	65	1306.67	–	–
IJTIHAD_V1	161	4344.53	96	2013.21	65	2331.32	–	–
IPROVER-QBF-BLOQER-2017	125	584.31	93	542.18	32	42.13	–	–
QELL_UNIT	104	9846.45	43	5202.41	61	4644.04	–	–
QELL_DEFAULT	103	8484.74	41	3581.03	62	4903.71	1	2.18
GHOSTQ-CEGAR	76	9162.22	45	6117.01	31	3045.21	–	–
NO-PREFIX-OPT-DEPQBF	74	8030.01	19	2090.97	55	5939.04	–	–
CUED1919_NL	73	9493.36	38	7359.17	35	2134.19	–	–
CUED1919_NNL	70	8972.15	35	6149.50	35	2822.65	–	–
IPROVER-QBF-2017	16	536.12	11	19.92	5	516.20	–	–

Table 20

Classification of the instances of the 2QBF Track of QBFEVAL'17. The table is organized as Table 8.

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
amba	4	2	2	0	158.73	0	2	0
arithmetic	5	5	5	0	75.14	0	5	0
cycle-sched	4	4	4	0	22.51	0	4	0
disjunctive_decomposition	5	5	4	1	2.37	0	4	1
driver	4	4	4	0	0.37	0	4	0
genbuf	4	0	0	0	–	0	0	0
HardwareFixpoint	86	80	37	43	902.36	6	67	7
hwmcc	4	4	4	0	358.08	0	4	0
irqlkeapcte	35	35	35	0	140.18	0	35	0
ltl2aig-comp	4	1	1	0	10.45	0	1	0
LTL2DBA	4	1	1	0	0.43	0	1	0
mult-matrix	4	3	3	0	4.16	0	3	0
RankingFunctions	37	37	37	0	1.80	0	37	0
Reduction-finding	82	70	41	29	2349.79	3	63	4
Selection-hard	10	8	8	0	497.68	0	5	3
sketch	9	8	0	8	440.88	0	8	0
Sorting_networks	26	23	7	16	446.64	0	23	0
terminator	39	39	5	34	34.20	0	39	0
toy	4	4	4	0	4.14	0	4	0
wgrowing	10	6	6	0	492.89	0	3	3
wmiforward	4	4	4	0	0.12	0	4	0

Table 21

Results of RQBF Track of QBFEVAL'17. The table is organized as Table 15.

Sol-ver	Solved		True		False		Unique	
	#	Time	#	Time	#	Time	#	Time
QBFRELAY-LIMITED-DEPQBF	82	13968.71	51	7089.27	31	6879.44	14	2461.27
RAREQS	74	7782.52	36	4698.57	38	3083.95	3	1455.88
CAQE_2017_V2	67	6684.96	44	6382.66	23	302.30	–	–
CAQE_2017_V3	66	8244.28	41	6750.28	25	1494.00	–	–
CAQE_2017_V1	66	8776.50	42	7039.92	24	1736.58	1	824.68
HERETIQ	58	6968.48	13	1754.23	45	5214.25	6	2363.68
QESTO	58	13790.20	35	5760.10	23	8030.10	2	755.09
PREFIX-OPT-DEPQBF	54	5187.86	35	1732.01	19	3455.85	–	–
QELL_UNIT	53	5423.12	18	1408.27	35	4014.85	–	–
QELL_DEFAULT	53	5692.99	18	1576.14	35	4116.85	–	–
QUTE_OPT500	51	2661.01	35	1808.04	16	852.97	–	–
QUTE_RANDOM	51	2670.65	35	1809.36	16	861.29	–	–
QUTE_DEFAULT	49	2111.22	32	1521.59	17	589.63	1	250.22
XB-QSTS_XBQSTS1.0	48	11071.49	39	7475.83	9	3595.66	–	–
XB-QSTS_XBQSTS2.0	48	11073.47	39	7477.41	9	3596.06	–	–
XB-QSTS_BQSTS2.0	47	10865.21	38	7438.76	9	3426.45	–	–
GHOSTQ-PG_CEGAR	41	5611.56	23	3031.75	18	2579.81	–	–
REV_QFUN	33	1737.27	17	493.79	16	1243.48	–	–
IJTIHAD_V2	33	3851.94	7	842.48	26	3009.46	–	–
GHOSTQ-CEGAR	32	4144.25	19	3025.97	13	1118.28	–	–
IJTIHAD_V1	31	1461.48	6	12.65	25	1448.83	–	–
AIGSolve	19	2489.14	12	1624.18	7	864.96	1	320.91
QBFRELAY	14	744.65	8	29.10	6	715.55	–	–
IProver-QBF-BLOQQER-2017	11	933.02	11	933.02	–	–	–	–
GHOSTQ-PG_PLAIN	11	1689.30	7	711.88	4	977.42	–	–
HQSPRE_SOLVER	9	5.62	8	5.61	1	0.01	–	–
DYNQBF-BLOQQER-HQSPRE	8	25.50	8	25.50	–	–	–	–
IProver-QBF-2017	4	165.02	4	165.02	–	–	–	–
CUED1919_NL	–	–	–	–	–	–	–	–
CUED1919_NNL	–	–	–	–	–	–	–	–

Table 22

Classification of the instances of the RQBF Track of QBFEVAL'17. The table is organized as Table 8.

Family	Overall				Time	Hardness		
	N	#	T	F		EA	ME	MH
CombinationalEquivalence	30	30	15	15	98.74	0	30	0
CounterFactual	31	11	0	11	2532.70	0	6	5
Model_instances	46	28	23	5	3006.88	0	27	1
PositionalGames_gttt	64	41	39	2	2558.15	0	29	12
PositionalGames_hex	49	13	10	3	2656.75	0	13	0
Q_2_2_3	50	0	0	0	–	0	0	0
Q_2_3	47	16	0	16	644.61	0	14	2
Q_3_3	50	0	0	0	–	0	0	0
Q_3_3_3	50	1	1	0	320.91	0	0	1
RobotsD2	5	5	5	0	775.05	0	3	2
Selection-hard	10	5	5	0	171.63	0	5	0
Strategic_Companies	40	19	0	19	3965.50	0	14	5
wgrowing	33	9	9	0	757.25	0	9	0

References

- [1] T. Balyo, A. Biere, M. Iser, C. Sinz, S.A.T. Race, SAT Race 2015, Artif. Intell. 241 (2016) 45–65.
- [2] T. Balyo, M.J.H. Heule, M. Jarvisalo, SAT competition 2016: recent developments, in: Proc. of the Thirty-First AAAI Conference on Artificial Intelligence, AAAI'17, AAAI Press, 2017, pp. 5061–5063.
- [3] D.R. Cok, D. Déharbe, T. Weber, The 2014 SMT competition, J. Satisf. Boolean Model. Comput. 9 (2014) 207–242.
- [4] P. Marin, M. Narizzano, L. Pulina, A. Tacchella, E. Giunchiglia, Twelve years of QBF evaluations: QSAT is pspace-hard and it shows, Fundam. Inform. 149 (1–2) (2016) 133–158.
- [5] L. Pulina, The ninth QBF solvers evaluation – preliminary report, in: Proc. of the 4th International Workshop on Quantified Boolean Formulas, QBF'16, in: CEUR Workshop Proc., CEUR-WS.org, vol. 1719, 2016, pp. 1–13.
- [6] M. Janota, C. Jordan, W. Klieber, F. Lonsing, M. Seidl, A. Van Gelder, The QBFGallery 2014: the QBF competition at the FLoC olympic games, J. Satisf. Boolean Model. Comput. 9 (2016) 187–206.
- [7] F. Lonsing, M. Seidl, A.V. Gelder, The QBF gallery: behind the scenes, Artif. Intell. 237 (2016) 92–114.
- [8] A. Stump, G. Sutcliffe, C. Tinelli, StarExec: a cross-community infrastructure for logic solving, in: Proc. of the 7th International Joint Conference on Automated Reasoning, IJCAR'14, in: Lecture Notes in Computer Science, vol. 8562, Springer, 2014, pp. 367–373.
- [9] Diptarama, K. Narisawa, A. Shinohara, Extension of generalized tic-tac-toe: p stones for one move, IPSJ J. 55 (2014) 2344–2352.

- [10] G. Amendola, F. Ricca, M. Truszczynski, Generating hard random Boolean formulas and disjunctive logic programs, in: *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, AAAI Press, 2017, pp. 532–538.
- [11] F. Slivovsky, S. Szeider, Soundness of Q-resolution with dependency schemes, *Theor. Comput. Sci.* 612 (2016) 83–101.
- [12] C. Peschiera, L. Pulina, A. Tacchella, U. Bubeck, O. Kullmann, I. Lynce, The seventh QBF solvers evaluation (QBFEVAL'10), in: *Proc. of the 13th International Conference on Theory and Applications of Satisfiability Testing, SAT'10*, in: *Lecture Notes in Computer Science*, vol. 6175, Springer, Berlin Heidelberg, 2010, pp. 237–250.
- [13] R. Brummayer, F. Lonsing, A. Biere, Automated testing and debugging of SAT and QBF solvers, in: *Proc. of the 13th International Conference on Theory and Applications of Satisfiability Testing, SAT'10*, in: *Lecture Notes in Computer Science*, vol. 6175, Springer, 2010, pp. 44–57.
- [14] H. Chen, Y. Interian, A model for generating random quantified Boolean formulas, in: *Proc. of 19th International Joint Conference on Artificial Intelligence, IJCAI'05*, 2005, pp. 66–71.
- [15] P.J. Rousseeuw, Silhouettes: a graphical aid to the interpretation and validation of cluster analysis, *J. Comput. Appl. Math.* 20 (1987) 53–65.
- [16] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, I.H. Witten, The weka data mining software: an update, *ACM SIGKDD Explor. Newsl.* 11 (1) (2009) 10–18.
- [17] F. Pigorsch, C. Scholl, Exploiting structure in an AIG based QBF solver, in: *Design, Automation and Test in Europe, DATE'09*, IEEE, 2009, pp. 1596–1601.
- [18] C. Scholl, F. Pigorsch, The QBF solver AIGSolve, in: *Proc. of the 4th Int. Workshop on Quantified Boolean Formulas, QBF 2016*, in: *CEUR Workshop Proc.*, CEUR-WS.org, vol. 1719, 2016, pp. 55–62.
- [19] F. Pigorsch, C. Scholl, An AIG-based QBF-solver using SAT for preprocessing, in: *Proc. of the 47th International Conference on Design Automation Conference, DAC'10*, IEEE, 2010, pp. 170–175.
- [20] I. Gent, E. Giunchiglia, M. Narizzano, A. Rowley, A. Tacchella, Watched data structures for QBF solvers, in: *Proc. of the Sixth International Conference on Theory and Applications of Satisfiability Testing, SAT'03*, in: *Lecture Notes in Computer Science*, vol. 2919, Springer Verlag, 2003, pp. 25–36.
- [21] E. Giunchiglia, M. Narizzano, A. Tacchella, Clause-term resolution and learning in quantified Boolean logic satisfiability, *Artif. Intell. Res.* 26 (2006) 371–416.
- [22] C.P. Gomes, B. Selman, H. Kautz, Boosting combinatorial search through randomization, in: *Proceedings of the Fifteenth National Conference on Artificial Intelligence and Tenth Innovative Applications of Artificial Intelligence Conference, AAAI'98/IAAI'98*, vol. 98, AAAI Press / The MIT Press, 1998, pp. 431–437.
- [23] K. Pipatsrisawat, A. Darwiche, A lightweight component caching scheme for satisfiability solvers, in: *Proc. of the 10th International Conference on Theory and Applications of Satisfiability Testing, SAT'07*, in: *Lecture Notes in Computer Science*, vol. 4501, Springer, 2007, pp. 294–299.
- [24] L. Zhang, S. Malik, Conflict driven learning in a quantified Boolean satisfiability solver, in: *Proc. of the International Conference on Computer Aided Design, ICCAD'02*, ACM / IEEE Computer Society, 2002, pp. 442–449.
- [25] L. Zhang, C.F. Madigan, M.W. Moskewicz, S. Malik, Efficient conflict driven learning in a Boolean satisfiability solver, in: *Proc. of the International Conference on Computer-Aided Design, ICCAD'01*, 2001, pp. 279–285.
- [26] E. Giunchiglia, P. Marin, M. Narizzano, sQueueBF: an effective preprocessor for QBFs based on equivalence reasoning, in: *Proc. of the 13th International Conference on Theory and Applications of Satisfiability Testing, SAT'10*, in: *Lecture Notes in Computer Science*, vol. 6175, Springer, 2010, pp. 85–98.
- [27] L. Pulina, A. Tacchella, A self-adaptive multi-engine solver for quantified Boolean formulas, *Constraints* 14 (1) (2009) 80–116.
- [28] M. Janota, J. Marques-Silva, Abstraction-based algorithm for 2QBF, in: *Proc. of the 14th International Conference on Theory and Applications of Satisfiability Testing, SAT'11*, in: *Lecture Notes in Computer Science*, vol. 6695, Springer, 2011, pp. 230–244.
- [29] A. Biere, F. Lonsing, M. Seidl, Blocked clause elimination for QBF, in: N. Bjørner, V. Sofronie-Stokkermans (Eds.), *Proc. of the 23rd International Conference on Automated Deduction, CADE'11*, in: *Lecture Notes in Computer Science*, vol. 6803, Springer, 2011, pp. 101–115.
- [30] T. Eiter, G. Gottlob, On the computational cost of disjunctive logic programming: propositional case, *Ann. Math. Artif. Intell.* 15 (3–4) (1995) 289–323.
- [31] M.N. Rabe, S.A. Seshia, Incremental determinization, in: *Proc. of the 19th International Conference on Theory and Applications of Satisfiability Testing, SAT'16*, in: *Lecture Notes in Computer Science*, vol. 9710, Springer, 2016, pp. 375–392.
- [32] M.N. Rabe, L. Tentrup, CAQE: a certifying QBF solver, in: *Proc. of the 15th Conference on Formal Methods in Computer-Aided Design, FMCAD'15*, 2015, pp. 136–143.
- [33] M. Narizzano, C. Peschiera, L. Pulina, A. Tacchella, Evaluating and certifying QBFs: a comparison of state-of-the-art tools, *AI Commun.* 22 (4) (2009) 191–210.
- [34] F. Lonsing, A. Biere, DepQBF: a dependency-aware QBF solver, *JSAT* 7 (2–3) (2010) 71–76.
- [35] R. Letz, Lemma and model caching in decision procedures for quantified Boolean formulas, in: *Proc. of the International Conference on Automated Reasoning With Analytic Tableaux and Related Methods, TABLEUX'02*, in: *Lecture Notes in Computer Science*, vol. 2381, Springer, 2002, pp. 160–175.
- [36] L. Zhang, S. Malik, Towards a symmetric treatment of satisfaction and conflicts in quantified Boolean formula evaluation, in: *Proc. of the Eighth International Conference on Principles and Practice of Constraint Programming, CP'02*, in: *Lecture Notes in Computer Science*, vol. 2470, 2002, pp. 200–215.
- [37] F. Lonsing, F. Bacchus, A. Biere, U. Egly, M. Seidl, Enhancing search-based QBF solving by dynamic blocked clause elimination, in: *Proc. of the 20th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning, LPAR'15*, in: *Lecture Notes in Computer Science*, vol. 9450, Springer, 2015, pp. 418–433.
- [38] F. Lonsing, U. Egly, Depqbf 6.0: a search-based QBF solver beyond traditional QCDC, in: *Proc. of the 26th International Conference on Automated Deduction, CADE 26*, in: *Lecture Notes in Computer Science*, vol. 10395, Springer, 2017, pp. 371–384.
- [39] A. Niemetz, M. Preiner, F. Lonsing, M. Seidl, A. Biere, Resolution-based certificate extraction for QBF, in: *Proc. of the 15th International Conference on Theory and Applications of Satisfiability Testing, SAT'12*, in: *Lecture Notes in Computer Science*, vol. 7317, 2012, pp. 430–435.
- [40] G. Charwat, S. Woltran, DynQBF: a dynamic programming-based QBF solver, <http://dbai.tuwien.ac.at/proj/decodyn/dynqbf>, 2016.
- [41] W. Klieber, S. Sapsa, S. Gao, E. Clarke, A non-prenex, non-clausal QBF solver with game-state learning, in: *Proc. of the 13th International Conference on Theory and Applications of Satisfiability Testing, SAT'10*, in: *Lecture Notes in Computer Science*, vol. 6175, Springer, 2010, pp. 128–142.
- [42] T. Balyo, F. Lonsing HordeQBF, A modular and massively parallel QBF solver, in: *Proc. of the 19th International Conference on Theory and Applications of Satisfiability Testing, SAT'16*, in: *Lecture Notes in Computer Science*, vol. 9710, Springer, 2016, pp. 531–538.
- [43] T. Balyo, P. Sanders, C. Sinz HordeSat, A massively parallel portfolio SAT solver, in: *Proc. of the 18th International Conference on Theory and Applications of Satisfiability Testing, SAT'15*, in: *Lecture Notes in Computer Science*, vol. 9340, Springer, 2015, pp. 156–172.
- [44] R. Wimmer, S. Reimer, P. Marin, B. Becker, HQSpre—an effective preprocessor for QBF and DQBF, in: *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, Springer, 2017, pp. 373–390.
- [45] R. Bloem, N. Braud-Santoni, V. Hadzic, QBF solving by counterexample-guided expansion, *CoRR*, arXiv:1611.01553 [abs].
- [46] K. Korovin, Inst-gen – a modular approach to instantiation-based automated reasoning, in: *Programming Logics – Essays in Memory of Harald Ganzinger*, in: *Lecture Notes in Computer Science*, vol. 7797, Springer, 2013, pp. 239–270.
- [47] M. Seidl, F. Lonsing, A. Biere, qbf2epr: a tool for generating EPR formulas from QBF, in: *Proc. of the Third Workshop on Practical Aspects of Automated Reasoning, PAAR'12*, in: *EPIC Series in Computing*, EasyChair, vol. 21, 2012, pp. 139–148.
- [48] C. Jordan, L. Kaiser, F. Lonsing, M. Seidl, MPIDepQBF: towards parallel QBF solving without knowledge sharing, in: *Proc. of the 17th International Conference on Theory and Applications of Satisfiability Testing, SAT'14*, in: *Lecture Notes in Computer Science*, vol. 8561, Springer, 2014, pp. 430–437.
- [49] A. Van Gelder, Primal and dual encoding from applications into quantified Boolean formulas, in: *Proc. of the 19th International Conference on Principles and Practice of Constraint Programming, CP'13*, in: *Lecture Notes in Computer Science*, vol. 8124, Springer, 2013, pp. 694–707.

- [50] K. Tu, T. Hsu, J.R. Jiang, QELL: QBF reasoning with extended clause learning and leveled SAT solving, in: Proc. of the 15th International Conference on Theory and Applications of Satisfiability Testing, SAT'15, in: Lecture Notes in Computer Science, vol. 9340, Springer, 2015, pp. 343–359.
- [51] M. Janota, J. Marques-Silva, Solving QBF by clause selection, in: Proc. of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI'15, AAAI Press, 2015, pp. 325–331.
- [52] T. Janhunen, S. Tasharrofi, E. Ternovska SAT-TO-SAT, Declarative extension of SAT solvers with new propagators, in: Proc. of the Thirtieth AAAI Conference on Artificial Intelligence, AAAI'16, AAAI Press, 2016, pp. 978–984.
- [53] B. Bogaerts, T. Janhunen, S. Tasharrofi, Solving QBF instances with nested SAT solvers, in: Proc. of the 2016 AAAI Beyond NP Workshop, in: AAAI Workshops, vol. WS-16-05, AAAI Press, 2016, pp. 307–313.
- [54] F. Lonsing, A. Biere, Failed literal detection for QBF, in: Proc. of the 14th International Conference on Theory and Applications of Satisfiability Testing, SAT'11, in: Lecture Notes in Computer Science, vol. 6695, Springer, 2011, pp. 259–272.
- [55] J. Devriendt, B. Bogaerts, M. Bruynooghe BreakIDGlucose, On the importance of row symmetry in SAT, in: Proc. 4th International Workshop on the Cross-Fertilization Between CSP and SAT, 2014, pp. 1–17.
- [56] L. Tentrup, Non-prenex QBF solving using abstraction, in: Proc. of the 19th International Conference on Theory and Applications of Satisfiability Testing, SAT'16, in: Lecture Notes in Computer Science, vol. 9710, Springer, 2016, pp. 393–401.
- [57] T. Peitl, F. Slivovsky, S. Szeider, Dependency learning for QBF, in: Proc. of the 20th International Conference on Theory and Applications of Satisfiability Testing, SAT'17, in: Lecture Notes in Computer Science, vol. 10491, Springer, 2017, pp. 298–313.
- [58] M. Janota, W. Klieber, J. Marques-Silva, E. Clarke, Solving QBF with counterexample guided refinement, in: Proc. of the 15th International Conference on Theory and Applications of Satisfiability Testing, SAT'12, in: Lecture Notes in Computer Science, vol. 7317, Springer, 2012, pp. 114–128.
- [59] M. Janota, W. Klieber, J. Marques-Silva, E. Clarke, Solving QBF with counterexample guided refinement, *Artif. Intell.* 234 (2016) 1–25.
- [60] L. Pulina, A. Tacchella, A structural approach to reasoning with quantified Boolean formulas, in: Proc. of the 21st International Joint Conference on Artificial Intelligence, IJCAI 2009, 2009, pp. 596–602.
- [61] M. Narizzano, L. Pulina, A. Tacchella, The QBFEVAL web portal, in: Proc. of the 10th European Conference on Logics in Artificial Intelligence, JELIA'06, in: Lecture Notes in Computer Science, vol. 4160, Springer Verlag, 2006, pp. 494–497.